

EXP NO : 1	8051 ASSEMBLY LANGUAGE EXPERIMENTS USING SIMULATOR

AIM:

To write and execute Assembly Language Program for Addition, Subtraction, Multiplication and Division of two 8-Bit numbers using EDSIM51 simulator.

SOFTWARE REQUIRED:

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	EDSIM51 Software	1

ALGORITHM:

1. Start the program.
2. Clear carry
3. Load 1st value in Accumulator (A Register)
4. Load 2nd value in B register
5. Add Accumulator and B register value and result store in the Accumulator
6. Store the resultant value in R0 register
7. Clear carry
8. Load 1st value in Accumulator (A Register)
9. Subtract Accumulator and B register value and result store in the Accumulator
10. Store the resultant value in R1 register
11. Clear carry
12. Load 1st value in Accumulator (A Register)
13. Multiply Accumulator and B register value and LSB value Stored in A Register ,MSB value Stored in B Register
14. Store the resultant value LSB in R2 Register and MSB in R3Register

15. Clear carry
16. Load 1st value in Accumulator (A Register)
17. Divide Accumulator and B register value and Quotient value Stored in A Register , Reminder value Stored in B Register
18. Store the resultant value Quotient in R4 Register and Reminder in R5

Register

19. Stop the program.

PROCEDURE:

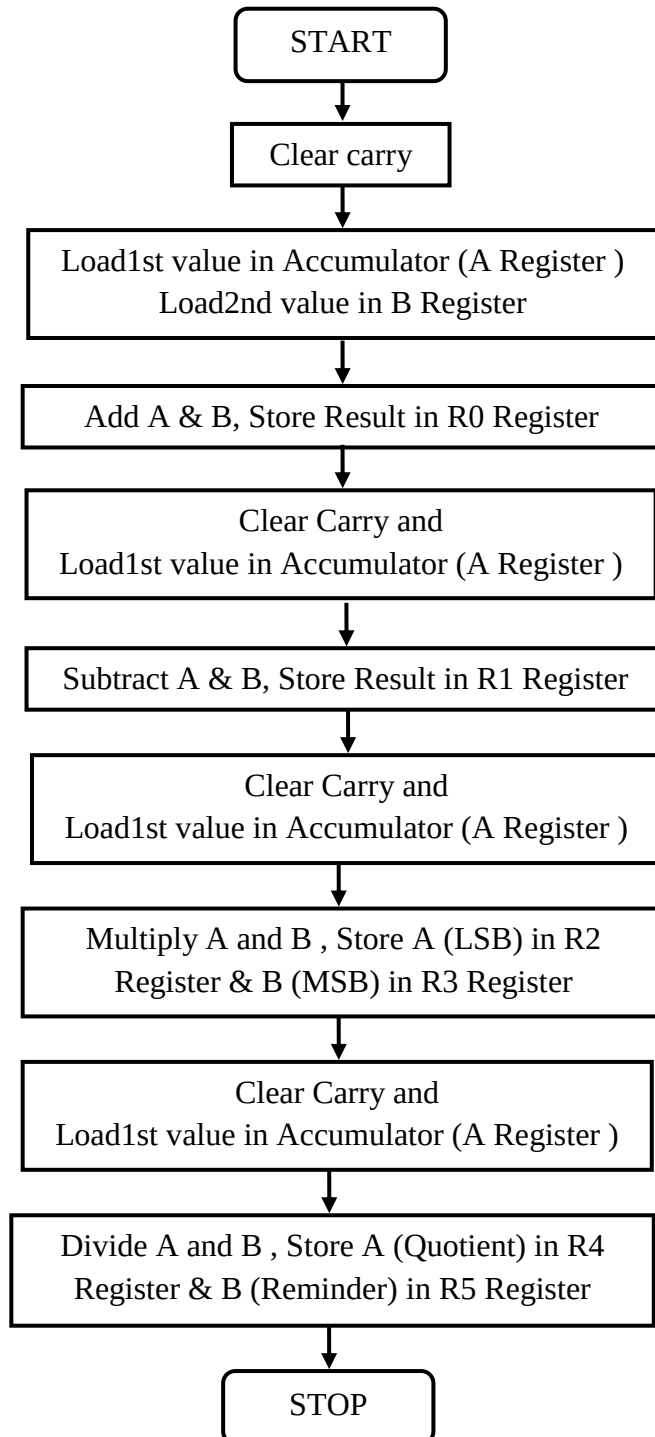
1. Open EDSIM51 Software
2. Type the assembly language in Programming Section
3. Click Assm Button to compile the program (If any error present click RST Button and debug the program)
4. Click Run Button to execute the program
5. Verify the result in Registers and Internal RAM section
6. Click Pause Button to Stop the program

PROGRAM:

PROGRAM	COMMENT
Clr C	Clear Carry
Mov A, #33h	Load 1st Data In Accumulator
Mov B, #22h	Load 2nd Data In B Register
Add A,B	Add A & B
Mov R0, A	Store Data In R0 Register
Clr C	Clear Carry
Mov A, #33h	Load 1st Data In Accumulator
Subb A,B	Subtract A & B
Mov R1, A	Store Data In R1 Register
Clr C	Clear Carry
Mov A, #33h	Load 1st Data In Accumulator
Mul Ab	Multiply A & B
Mov R2, A	Store Lsb Data In R2 Register
Mov R3, B	Store Msb Data In R3 Register
Clr C	Clear Carry
Mov A, #33h	Load 1st Data In Accumulator
Div Ab	Divide A & B

Mov R4, A	Store Quotient Data In R4
	Register
Mov R5, B	Store Reminder Data In R5
	Register
End	Stop Program

FLOW CHART



RESULT:

Thus the Assembly Language Programs for Addition, Subtraction, Multiplication and Division of two 8-Bit numbers was simulated.

EXP NO : 2	TEST DATA TRANSFER BETWEEN REGISTERS AND MEMORY

AIM:

To write and execute Assembly Language Program for Circular Logic Rotate Right, Circular Logic Rotate Left and Swap Operations of 8-Bit number using EDSIM51 simulator.

REQUIRED ITEMS:

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	EDSIM51 Software	1

ALGORITHM:

1. Start the program.
2. Clear carry
3. Load 1st value in Accumulator (A Register)
4. Rotate Right Logic in Accumulator and result store in the Accumulator
6. Store the resultant value in R0 register
7. Load 1st value in Accumulator (A Register)
8. Rotate Left Logic in Accumulator and result store in the Accumulator
9. Store the resultant value in R1 register
10. Load 1st value in Accumulator (A Register)
11. Swap Logic in Accumulator and result store in the Accumulator
12. Store the resultant value in R2 Register
14. Stop the program.

PROCEDURE:

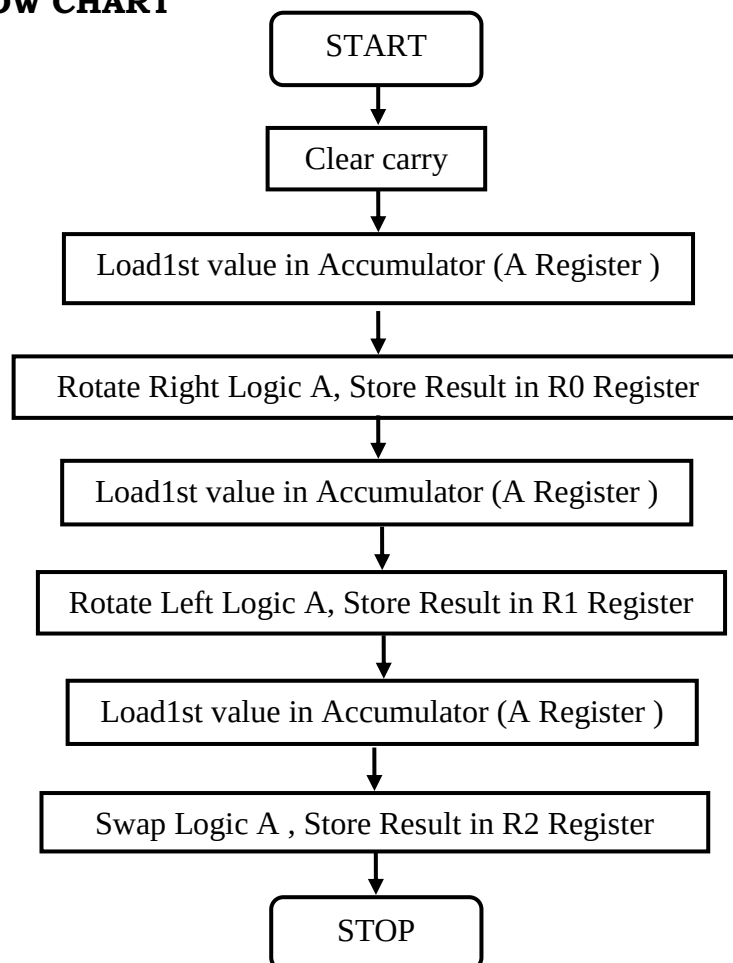
1. Open EDSIM51 Software
2. Type the assembly language in Programming Section
3. Click Assm Button to compile the program (If any error present click RST Button and debug the program)

4. Click Run Button to execute the program
5. Verify the result in Registers and Internal RAM section
6. Click Pause Button to stop the program

PROGRAM

PROGRAM	COMMENT
clr c	Clear Carry
mov a, #33h	Load 1st Data in Accumulator
rr a	Rotate Right Logic A
mov r0, a	Store Data in R0 Register
mov a, #33h	Load 1st Data in Accumulator
rl a	Rotate Right Logic A
mov r1, a	Store Data in R1 Register
mov a, #33h	Load 1st Data in Accumulator
swap a	Swap Logic A
mov r2, a	Store Data in R2 Register
end	Stop Program

FLOW CHART



RESULT:

Thus the data transfer between registers and memory by using simulator was successfully tested.

EXPNO : 3

PERFORM ALU OPERATIONS

AIM:

To write and execute Arithmetic and Logical Operations of two 8-Bit numbers using EDSIM51 simulator

REQUIRED ITEMS:

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	EDSIM51 Software	1

ALGORITHM:

1. Start the program.
2. Clear carry
3. Load 1st value in Accumulator (A Register)
4. Load 2nd value in B register
5. AND Logic between Accumulator and B register value and result store in the Accumulator
6. Store the resultant value in R0 register
7. Load 1st value in Accumulator (A Register)
8. OR Logic between Accumulator and B register value and result store in the Accumulator
9. Store the resultant value in R1 register
10. Load 1st value in Accumulator (A Register)
11. XOR Logic between Accumulator and B Register value and result store in the Accumulator
12. Store the resultant value in R2 Register
13. Load 1st value in Accumulator (A Register)

14. NOT Logic between Accumulator value and result store in the Accumulator
15. Store the resultant value in R3 Register
16. Load 1st value in Accumulator (A Register)
17. Perform Increment Operation in Accumulator value and result store in the Accumulator
18. Store the resultant value in R4 register
19. Load 1st value in Accumulator (A Register)
20. Perform Decrement Operation in Accumulator value and result store in the Accumulator
21. Store the resultant value in R5 register
22. Stop the program.

PROCEDURE:

1. Open EDSIM51 Software
2. Type the assembly language in Programming Section
3. Click Assm Button to compile the program (If any error present click RST Button and debug the program)
4. Click Run Button to execute the program
5. Verify the result in Registers and Internal RAM section
6. Click Pause Button to stop the program

PROGRAM

PROGRAM	COMMENT
clr c	Clear Carry
mov a, #33h	Load 1st Data in Accumulator
mov b, #22h	Load 2nd Data in B Register
anl a,b	AND Logic between A & B
mov r0, a	Store Data in R0 Register
mov a, #33h	Load 1st Data in Accumulator
orl a,b	OR Logic between A & B
mov r1, a	Store Data in R1 Register
mov a, #33h	Load 1st Data in Accumulator
xrl a,b	XOR Logic between A & B
mov r2, a	Store LSB Data in R2 Register
mov a, #33h	Load 1st Data in Accumulator
cpl a	NOT Logic in A
mov r3, a	Store Data in R3 Register
mov a, #33h	Load 1st Data in Accumulator
inc a	Increment in A value

```

mov r4, a
mov a, #33h
dec a
mov r5, a
end

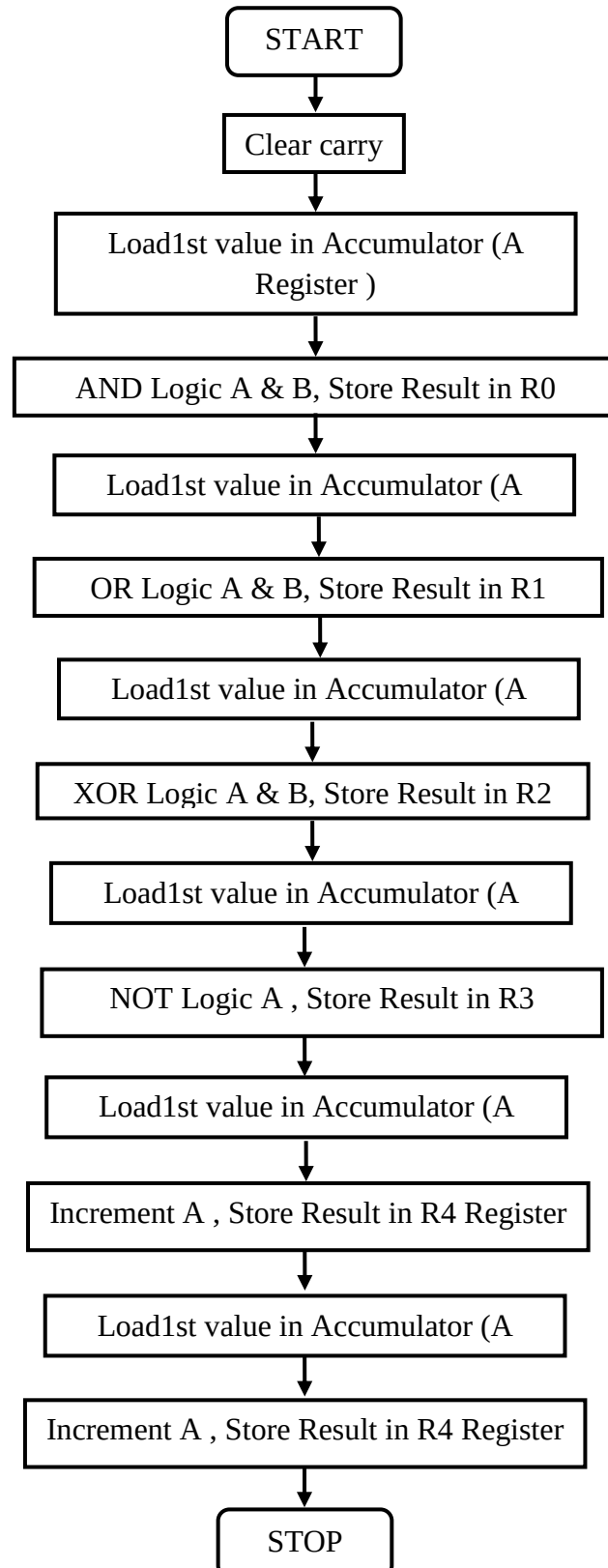
```

```

Store Data in R0 register
Load 1st Data in Accumulator
Decrement in A value
Store Data in R1 register
Stop Program

```

FLOW CHART



RESULT:

Thus the Arithmetic and Logical Operations of two 8-Bit numbers using EDSIM51 simulator was verified successfully.

EXPNO : 4

ARITHMETIC PROGRAMS USING EMBEDDED C.

AIM:

To write and execute Embedded Language Program for Addition, Subtraction, Multiplication, Modulation and Division of two 8-Bit numbers using Keil uVision5.

REQUIRED ITEMS

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	Keil uVision5 Software	1

PROCEDURE:

1. Start the Keil software. Go to the **Project > New Project** then choose a location to store your program, and give a name and **Save**.
2. Now in the next window select the device from different manufacturers. We are selecting **Microchip**, and then by expanding we are selecting **AT89C51** device and click ok
3. Now go to the **New** in the menu and select **New**. It will open a new editor to write code.
4. Go to the save option and save the program file with .asm extension.
5. Write the code for 8051 Microcontroller. (Here we are using a arithmetic codes)
6. Now from the left panel, select **Source Group 1**, and **Add Existing Files to Group 'Source Group 1'**. Then select the program (c file) then **add and close**
7. Now go to the **Project > Build Target** to build the project. If there is some error the building will be failed, after correcting the errors it can be build.
8. Now click on the **Target1** from the left panel and select **Options for Target 'Target1'**. Then set the xtal (MHz) value to 11.0592. Check mark

on the Use On Chip ROM. Then go to the output tab. In this tab check **Create Hex File**, and click OK. Then build it again.

PROGRAM:

Addition

```
ORG 00H
SJMP START
START :Mov a,#50H
Mov b,#30h
add a,b
exit:sjmp exit
END
```

Subtraction

```
ORG 00H
SJMP START
START :Mov a,#30H
Mov b,#20h
subb a,b
exit:sjmp exit
END
```

Multiplication

```
ORG 00H
SJMP START
START :Mov a,#35H
Mov b,#32h
mul ab
exit:sjmp exit
END
```

Division

```
ORG 00H
SJMP START
START :Mov a,#35H
Mov b,#32h
div ab
exit:sjmp exit
END
```

RESULT:

Thus the Basic and arithmetic Programs Using Embedded C was Executed successfully

EXPNO: 5

**INTRODUCTION TO ARDUINO PLATFORM AND
PROGRAMMING**

AIM

To write a Program for LED Array with Arduino Using Arduino platform

REQUIRED ITEMS

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	CS3691 IOT TRAINER KIT • Arduino Nano unit • LED Array unit	1
3	Jumper Wires	As Required
4	USB Cable	1

ALGORITHM:

1. Start the program
2. Define GPIO Pins
3. Define GPIO Pin Mode
4. if Infinite loop True:
 - ON LED_1
 - Delay 1 Second
 - ON LED_2
 - Delay 1 Second
 - ON LED_3
 - Delay 1 Second
 - ON LED_4
 - Delay 1 Second
 - OFF LED_1
 - OFF LED_2
 - OFF LED_3
 - OFF LED_4
 - Delay 1 Second

5. else Stop Program

PROCEDURE:

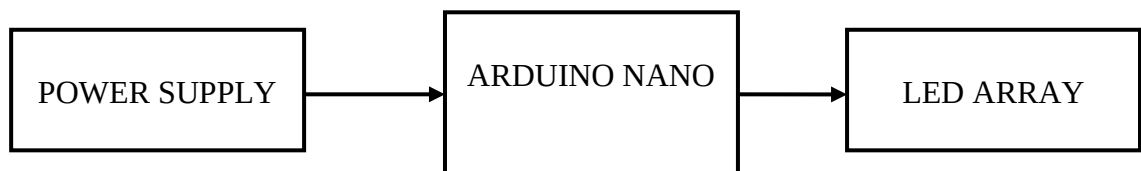
1. Open **ARDUINO IDE** Software
 2. Click **File >> New** or **CTRL+N** to create New Project.
 3. To Select Board Click **Tools >> Board > Arduino AVR Boards >> Arduino Nano**
 4. To select Processor Click **Tools >> Processor >> ATmega328P (Old Bootloader)**
 5. To select Programmer Click **Tools >> Programmer >> AVRISP mkII**
 6. Click **File >> Save** or **CTRL+S** to Save Project where we Desire.
 7. Type the Program.
 8. Click **Sketch >> Verify / Compile** or **CTRL+R** to compile the program (If any error present debug the program)
 9. Connect the Arduino Nano with PC / Laptop by USB Cable
 10. Select the COM PORT by Click **Tools >> Port** and Select Arduino's COM PORT Number.
- Note : If COM PORT Number is Don't Know:
- Right Click My Computer Icon and Select Properties
 - Click Device Manager
 - Expand Ports (COM & LPT). COM PORT Number will be shown
11. Click **Sketch >> Upload** or **CTRL+U** to upload the program to Arduino Nano.
 12. Switch On the corresponding Units we need and Verify the Program

PROGRAM

PROGRAM	COMMENT
int led1 = 2;	LED_1 connected with D2
int led2 = 3;	LED_2 connected with D3
int led3 = 4;	LED_3 connected with D4
int led4 = 5;	LED_4 connected with D5
void setup()	Set up Loop Start
{	
pinMode(led1 , OUTPUT);	Configure LED_1 as Output
pinMode(led2 , OUTPUT);	Configure LED_2 as Output
pinMode(led3 , OUTPUT);	Configure LED_3 as Output

pinMode(led4 , OUTPUT);	Configure LED_4 as Output
}	Set up Loop Close
void loop()	Infinite Loop Start
{	
digitalWrite(led1, HIGH);	LED_1 ON
delay(1000);	Delay 1 Second
digitalWrite(led2, HIGH);	LED_2 ON
delay(1000);	Delay Second
digitalWrite(led3, HIGH);	LED_3 ON
delay(1000);	Delay 1 Second
digitalWrite(led4, HIGH);	LED_4 ON
delay(1000);	Delay 1 Second
digitalWrite(led1, LOW);	LED_1 OFF
digitalWrite(led2, LOW);	LED_2 OFF
digitalWrite(led3, LOW);	LED_3 OFF
digitalWrite(led4, LOW);	LED_4 OFF
delay(1000);	Delay 1 Second
}	Infinite Loop Close

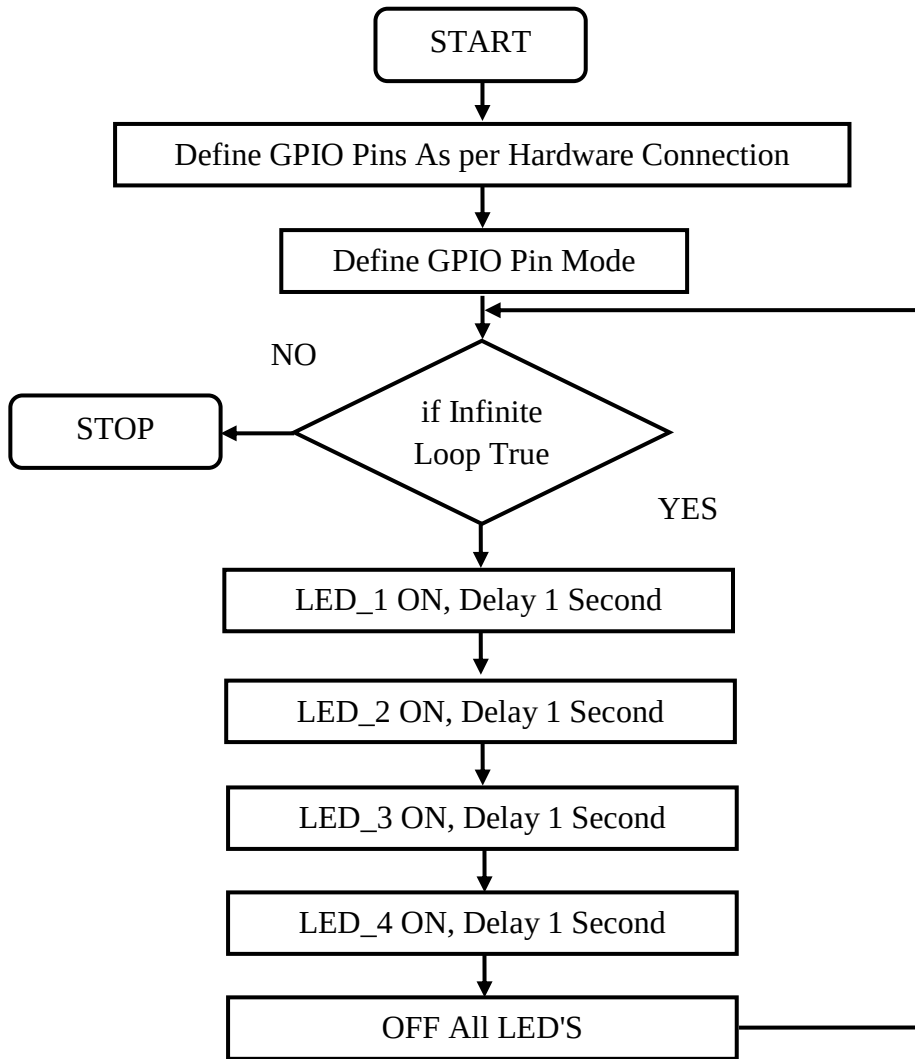
BLOCK DIAGRAM



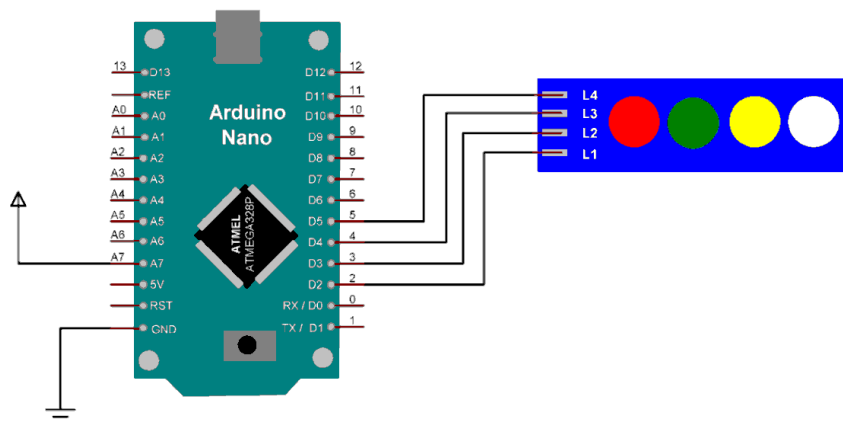
CONNECTION TABLE

S.NO	CONNECTION		PIN MODE
	COMPONENT	ARDUINO	
1	VCC	5V	POWER +
2	GND	GND	POWER -
3	LED_1 (L1)	D2	OUTPUT
4	LED_2 (L2)	D3	OUTPUT
5	LED_3 (L3)	D4	OUTPUT
6	LED_4 (L4)	D5	OUTPUT

FLOW CHART



CIRCUIT DIAGRAM



RESULT:

Thus the LED array blinking with arduino Programming was successfully executed

EXPNO: 6	DIFFERENT COMMUNICATION METHODS WITH IOT DEVICES

AIM:

To interface the Bluetooth with Arduino using Arduino IDE

REQUIRED ITEMS

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	CS3691 IOT TRAINER KIT • Arduino Nano Unit • HC-05 Bluetooth Unit	1
3	Jumper Wires	As Required
4	USB Cable	1

ALGORITHM:

1. Start the program
2. Variable declaration
3. Initialize Bluetooth
4. Send **Welcome Message**
5. if Infinite loop True:
 - if Bluetooth Data Available
 - Receive Data And Send Again to Bluetooth Module
6. else
 - Stop Program

PROCEDURE:

1. Open **ARDUINO IDE** Software
2. Click **File >> New** or **CTRL+N** to create New Project.
3. To Select Board Click **Tools >> Board > Arduino AVR Boards >> Arduino Nano**

4. To select Processor Click **Tools >> Processor >> ATmega328P (Old Bootloader)**
 5. To select Programmer Click **Tools >> Programmer >> AVRISP mkII**
 6. Click **File >> Save** or **CTRL+S** to Save Project where we Desire.
 7. Type the Program.
 8. Click **Sketch >> Verify / Compile** or **CTRL+R** to compile the program (If any error present debug the program)
 9. Connect the Arduino Nano with PC / Laptop by USB Cable
 10. Select the COM PORT by Click **Tools >> Port** and Select Arduino's COM PORT Number.
- Note : If COM PORT Number is Don't Know:
- Right Click My Computer Icon and Select Properties
 - Click Device Manager
 - Expand Ports (COM & LPT). COM PORT Number will be shown
11. Click **Sketch >> Upload** or **CTRL+U** to upload the program to Arduino Nano.
 12. Switch On the corresponding Units we need and Verify the Program.

PROGRAM

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27,16,2);
int led1 = 2;
int led2 = 3;
int led3 = 4;
int led4 = 5;
int l1=0;
int l2=0;
int l3=0;
int l4=0;
char data;
void setup()
{
    Serial.begin(9600);
    pinMode(led1 , OUTPUT);
    pinMode(led2 , OUTPUT);
    pinMode(led3 , OUTPUT);
    pinMode(led4 , OUTPUT);
    lcd.begin();
    lcd.backlight();
}
void loop()
{
```

```

if (Serial.available())
{
data = Serial.read();
Serial.print(data);
if(data=='1')
{
digitalWrite(led1, HIGH);
l1=1;
}
else if(data=='2')
{
digitalWrite(led1, LOW);
l1=0;
}
else if(data=='3')
{
digitalWrite(led2, HIGH);
l2=1;
}
else if(data=='4')
{
digitalWrite(led2, LOW);
l2=0;
}
else if(data=='5')
{
digitalWrite(led3, HIGH);
l3=1;
}
else if(data=='6')
{
digitalWrite(led3, LOW);
l3=0;
}
else if(data=='7')
{
digitalWrite(led4, HIGH);
l4=1;
}
else if(data=='8')
{
digitalWrite(led4, LOW);
l4=0;
}
else if(data=='9')
{
digitalWrite(led1, HIGH);
digitalWrite(led2, HIGH);
digitalWrite(led3, HIGH);
digitalWrite(led4, HIGH);
l1=1;l2=1;l3=1;l4=1;
}
}

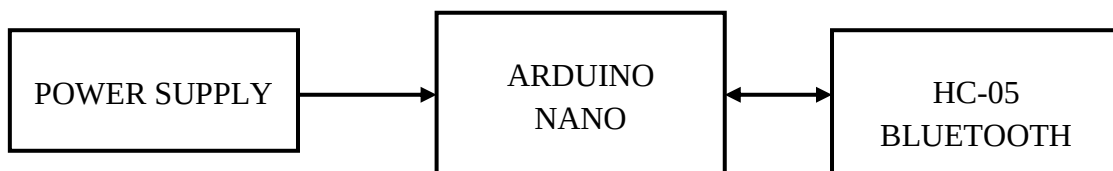
```

```

else
{
digitalWrite(led1, LOW);
digitalWrite(led2, LOW);
digitalWrite(led3, LOW);
digitalWrite(led4, LOW);
l1=0;l2=0;l3=0;l4=0;
}
}
}
lcd.clear();
lcd.setCursor(0,0);
if(l1==1)
{
lcd.print("L1: ON  |");
}
else
{
lcd.print("L1: OFF |");
}
if(l2==1)
{
lcd.print("L2: ON  |");
}
else
{
lcd.print("L2: OFF |");
}
lcd.setCursor(0,1);
if(l3==1)
{
lcd.print("L3: ON  |");
}
else
{
lcd.print("L3: OFF |");
}
if(l4==1)
{
lcd.print("L4: ON  |");
}
else
{
lcd.print("L4: OFF |");
}
delay(2000);
}

```

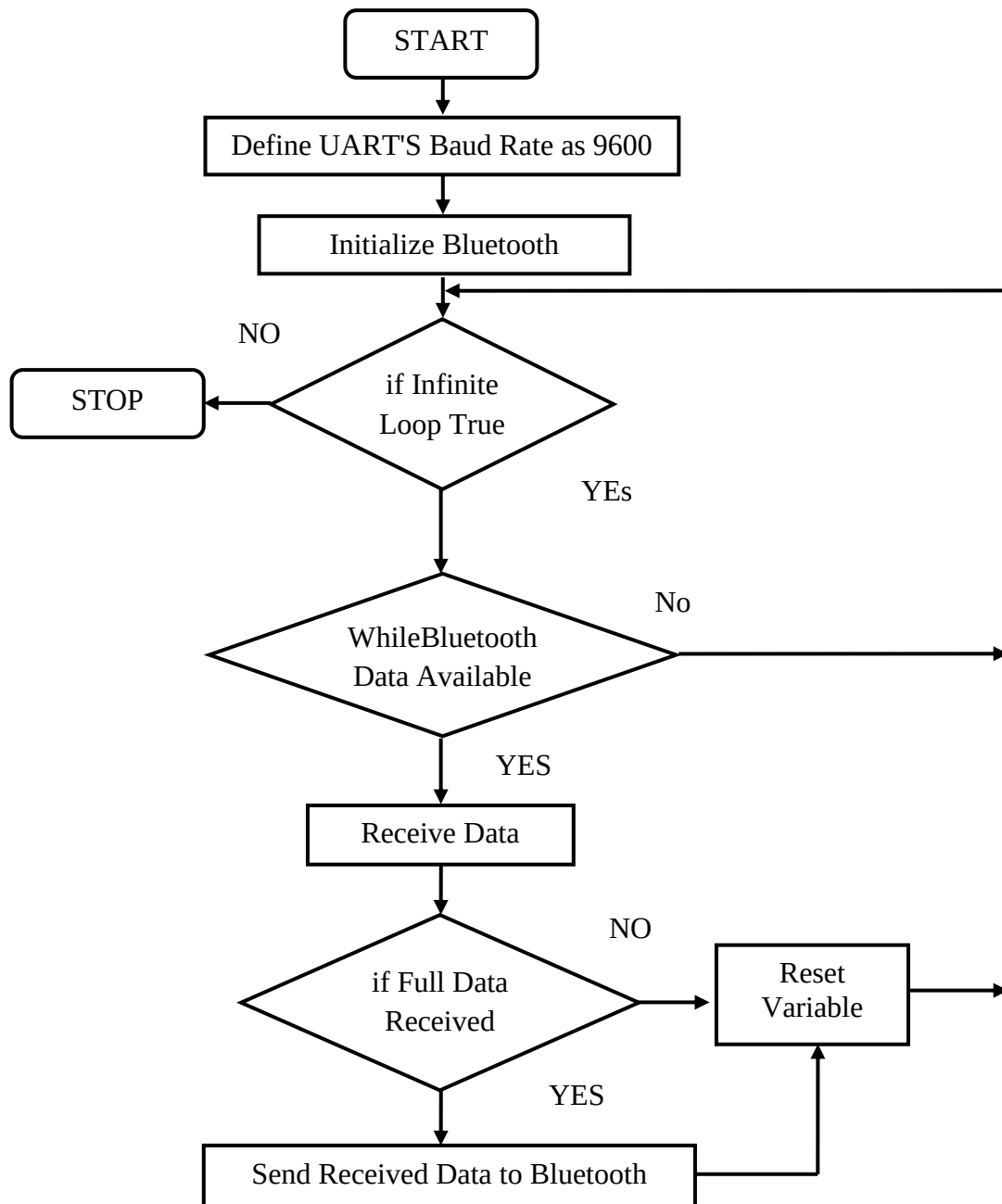
BLOCK DIAGRAM



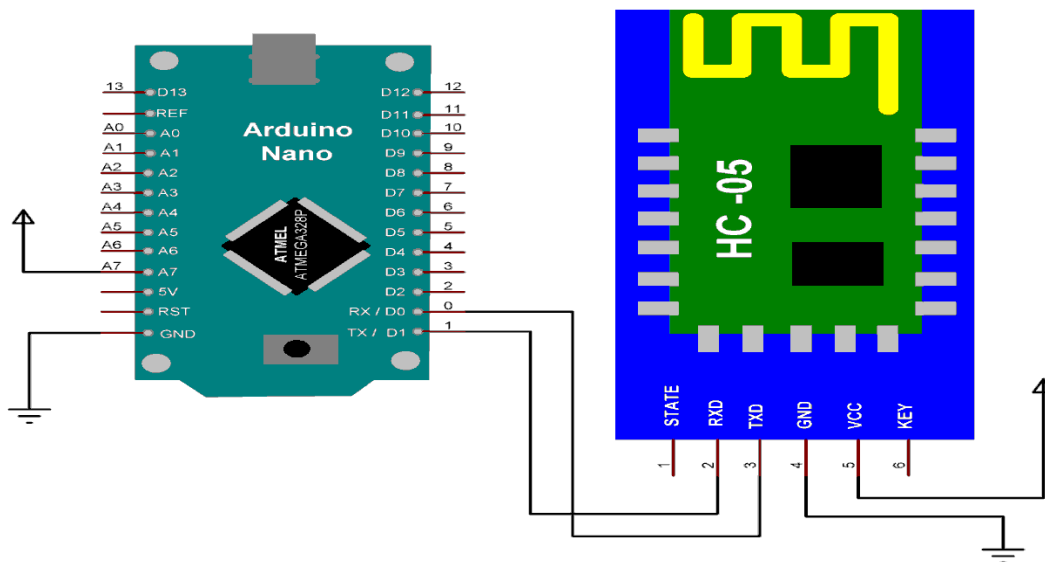
CONNECTION TABLE

S.NO	CONNECTION		PIN MODE
	COMPONENT	ARDUINO	
1	VCC	5V	POWER +
2	GND	GND	POWER -
3	HC-05_TX	RX	DATA RECIEVE
4	HC-05_RX	TX	DATA TRANSMIT

FLOWCHART:



CIRCUIT DIAGRAM



RESULT:

Thus the Different Communication Methods with IoT Devices was Executed Successfully

EXPNO: 7	INTRODUCTION TO RASPBERRY PI PLATFORM AND PYTHON PROGRAMMING

AIM:

To interface the 1X4 LED array with Raspberry Pi using
Thoonny Python IDE

REQUIRED ITEMS

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	CS3691 IOT TRAINER KIT • Raspberry Pi unit • LED Array unit	1
3	Jumper Wires	As Required
4	USB Cable	1

ALGORITHM:

1. Start the program
2. Define GPIO Pins
3. Define GPIO Pin Mode
4. if Infinite loop True:
 - ON LED_1
 - Delay 1 Second
 - ON LED_2
 - Delay 1 Second
 - ON LED_3
 - Delay 1 Second
 - ON LED_4
 - Delay 1 Second
 - OFF LED_1
 - OFF LED_2
 - OFF LED_3
 - OFF LED_4
 - Delay 1 Second
5. else
 - Stop Program

PROGRAM

PROGRAM	COMMENT
import RPi.GPIO as GPIO	import GPIO library
import time	import Time Library
GPIO.setwarnings(False)	Warning Off for GPIO pins
LED_PIN1 = 18	LED_1 connected with GPIO 18
LED_PIN2 = 23	LED_2 connected with GPIO 23
LED_PIN3 = 24	LED_3 connected with GPIO 24
LED_PIN4 = 25	LED_4 connected with GPIO 25
GPIO.setmode(GPIO.BCM)	Define as Physical numbering
GPIO.setup(LED_PIN1, GPIO.OUT)	Configure LED_1 as Output
GPIO.setup(LED_PIN2, GPIO.OUT)	Configure LED_2 as Output
GPIO.setup(LED_PIN3, GPIO.OUT)	Configure LED_3 as Output
GPIO.setup(LED_PIN4, GPIO.OUT)	Configure LED_4 as Output
while True:	Infinite Loop Start
GPIO.output(LED_PIN1, GPIO.HIGH)	LED_1 ON
time.sleep(1)	Delay 1 Second
GPIO.output(LED_PIN2, GPIO.HIGH)	LED_2 ON
time.sleep(1)	Delay Second
GPIO.output(LED_PIN3, GPIO.HIGH)	LED_3 ON
time.sleep(1)	Delay 1 Second
GPIO.output(LED_PIN4, GPIO.HIGH)	LED_4 ON
time.sleep(1)	Delay 1 Second
GPIO.output(LED_PIN4, GPIO.LOW)	LED_1 OFF
GPIO.output(LED_PIN3, GPIO.LOW)	LED_2 OFF
GPIO.output(LED_PIN2, GPIO.LOW)	LED_3 OFF
GPIO.output(LED_PIN1, GPIO.LOW)	LED_4 OFF
except:	Infinite Loop Close
GPIO.cleanup()	Reset GPIO Pins

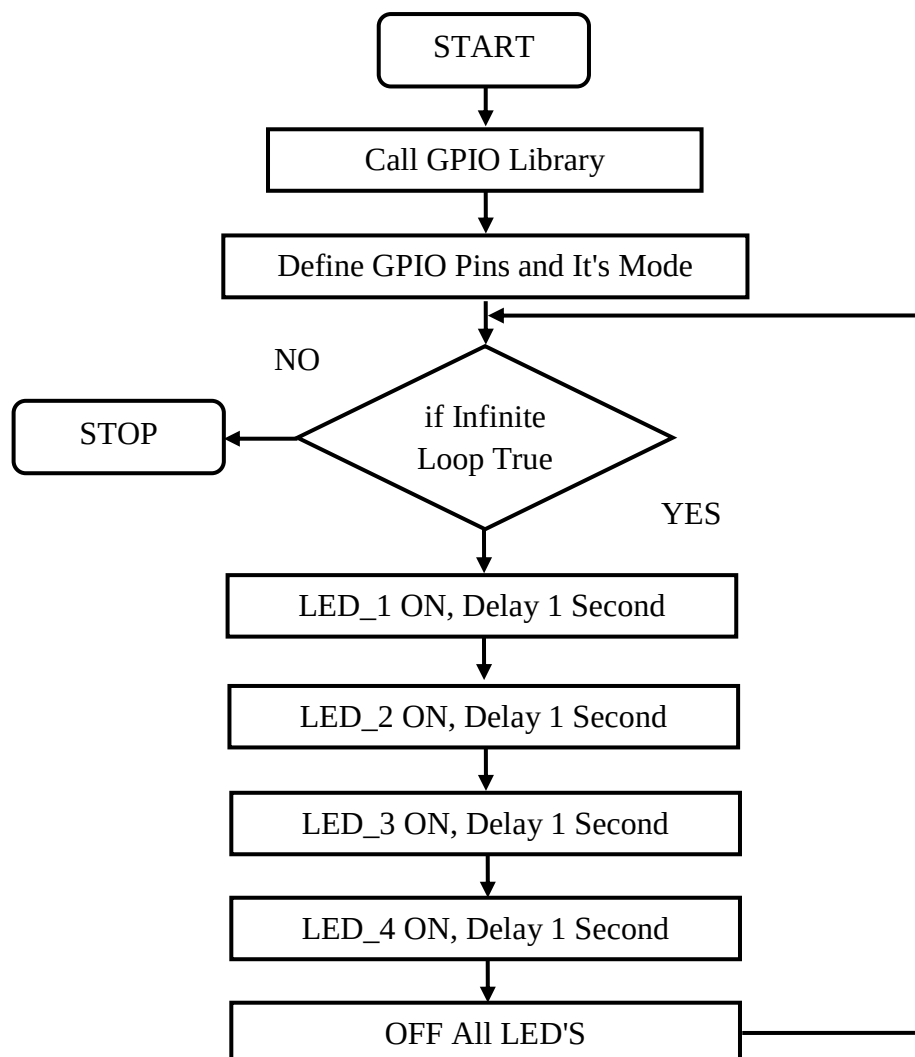
CONNECTION TABLE

S.NO	CONNECTION		PIN MODE
	COMPONENT	RASPBERRY PI	
1	VCC	5V	POWER +
2	GND	GND	POWER -
3	LED_1 (L1)	GPIO 18	OUTPUT
4	LED_2 (L2)	GPIO 23	OUTPUT
5	LED_3 (L3)	GPIO 24	OUTPUT
6	LED_4 (L4)	GPIO 25	OUTPUT

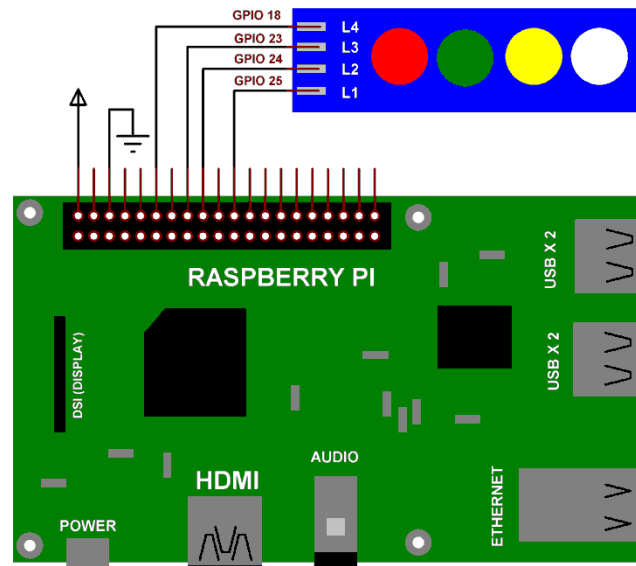
BLOCK DIAGRAM



FLOW CHART



CIRCUIT DIAGRAM



TERMINAL COMMAND LINE

1. `sudo apt-get update`
2. `sudo apt-get upgrade`
3. `sudo apt-get install rpi.gpio`

PROCEDURE:

1. Start **Thonny** by clicking on the **Raspberry Pi icon** followed by **Programming > Thonny Python IDE**.
2. Write your program in the top pane, click **File > Save as...** to save it,
3. Make hardware connections
4. Click **Run>** Run current script to execute the program. Output will appear in the bottom interpreter pane.

RESULT:

Thus the LED array interfacing with Raspberry pi was verified successfully

EXP NO : 8

INTERFACING SENSORS WITH RASPBERRY PI

Interface DHT11 Sensor and I2C LCD with Raspberry Pi

AIM:

To interface the DHT11 Sensor and I2C LCD with Raspberry Pi using Thonny Python IDE

REQUIRED ITEMS

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	CS3691 IOT TRAINER KIT • Raspberry Pi Unit • I2C LCD Unit • DHT 11 Sensor Unit	1
3	Jumper Wires	AS Required
4	USB Cable	1

ALGORITHM:

1. Start the program
2. Call I2C LCD Library and DHT11 Sensor Library
3. Define GPIO Pins
4. Variable Declaration
5. Initiate LCD
6. if Infinite loop True:
 - Read Temperature and Humidity Value
 - Clear LCD
 - Set LCD Cursor (0,0) Position 0 and Column 1
 - Print **TEMP: VALUE**
 - Set LCD Cursor (0,1) Position 0 and Column 2
 - Print **HUMI: VALUE**

Delay 3 Second

5. elseStop Program

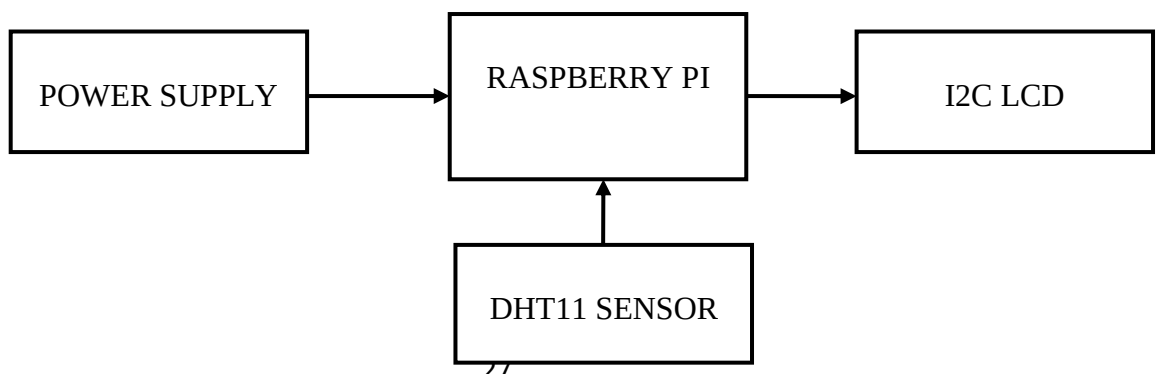
PROGRAM

```
import lcddriver
from time import *
lcd = lcddriver.lcd()
import Adafruit_DHT
import time
DHT_SENSOR = Adafruit_DHT.DHT11
DHT_PIN = 21
lcd lcd_display_string("RPSIT", 1)
lcd lcd_display_string("IT", 2)
sleep(1)
while True:
    humidity, temperature = Adafruit_DHT.read(DHT_SENSOR, DHT_PIN)
    if humidity is not None and temperature is not None:
        print("Temp={0:0.1f}C Humidity={1:0.1f}%".format(temperature, humidity))
        lcd lcd_clear()
        lcd lcd_display_string("Temp: %d%s C" % (temperature, chr(223)), 1)
        lcd lcd_display_string("Humidity: %d %% " % humidity, 2)
    else:
        print("DATA READING....");
        lcd lcd_clear()
        lcd lcd_display_string("DATA READING...", 1)
        lcd lcd_display_string("    ****" , 2)
    time.sleep(3);
```

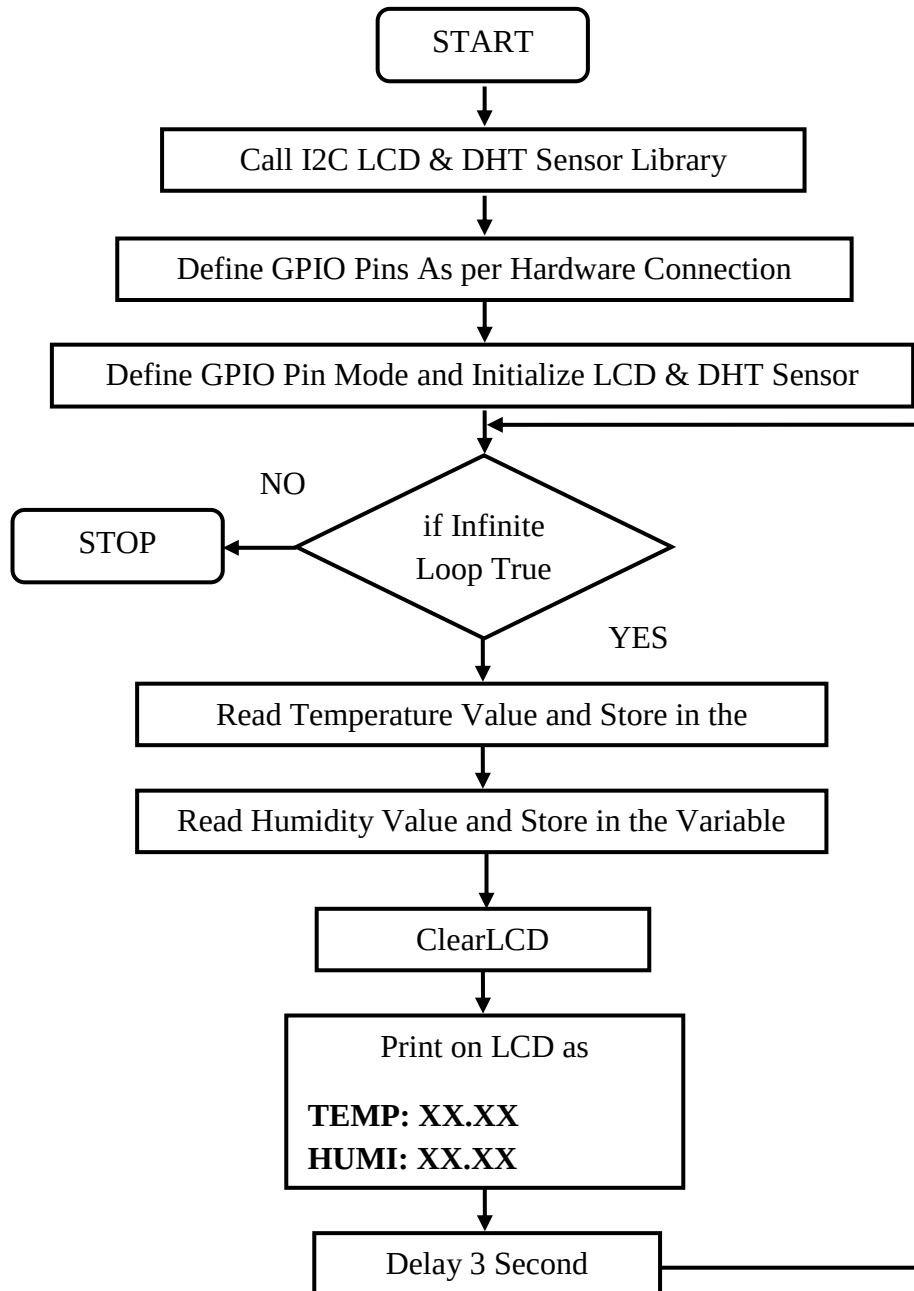
CONNECTION TABLE

S.NO	CONNECTION		PIN MODE
	COMPONENT	RASPBERRY PI	
1	VCC	5V	POWER +
2	GND	GND	POWER -
3	LCD (SDA)	SPIO 02	Serial Data
4	LCD (SCL)	GPIO 03	Serial Clock
5	DHT11	GPIO 25	Digital Input

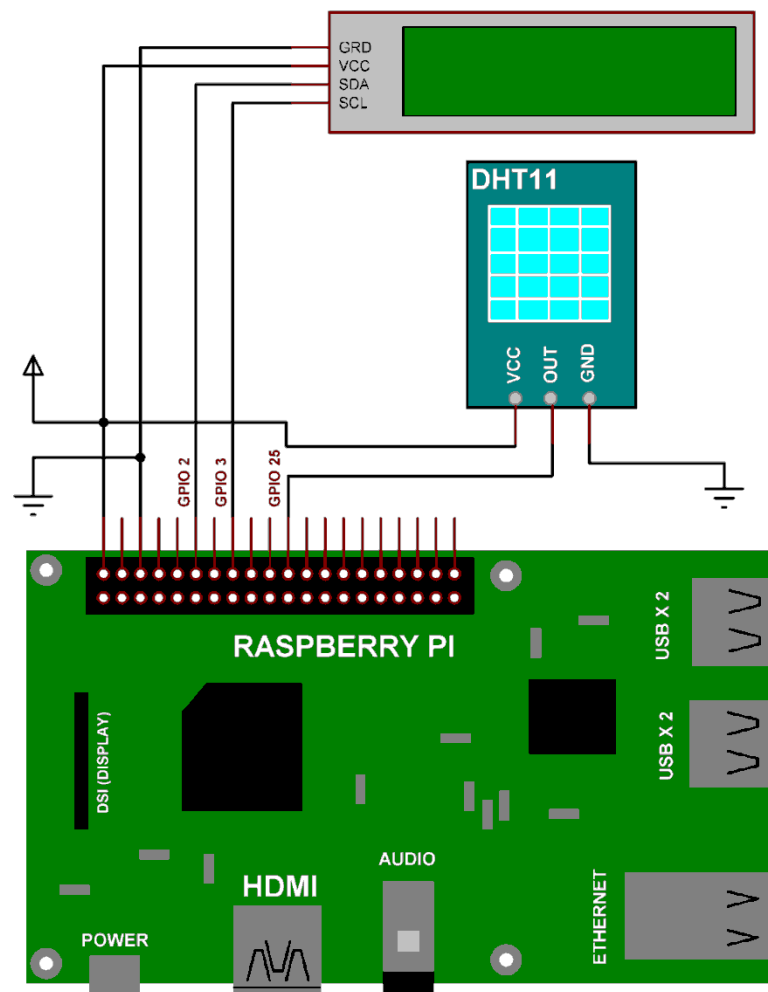
BLOCK DIAGRAM



FLOW CHART



CIRCUIT DIAGRAM



TERMINAL COMMAND LINE

1. `sudo apt-get update`
2. `sudo apt-get upgrade`
3. `sudo raspi-config`
4. Select Interfacing Options > I2C.
5. Select Yes when prompted to enable the I2C interface.
6. Select Yes when prompted to automatically load the I2C kernel module.
7. Select Finish.
8. `i2cdetect -y 1`
9. `sudo raspi-config`
10. Select Interfacing Options > 1 WIRE.
11. Select Yes when prompted to enable the 1 WIRE interface.

12. Select Yes when prompted to automatically load the 1 WIRE kernel module.
13. Select Finish.

PROCEDURE:

1. Start **Thonny** by clicking on the **Raspberry Pi icon** followed by **Programming > Thonny Python IDE**.
2. Write your program in the top pane, click **File > Save** as... to save it,
3. Make hardware connections
4. Click **Run>** Run current script to execute the program. Output will appear in the bottom interpreter pane.

RESULT:

Thus the temperature Sensor was Successfully Interface with Raspberry Pi

EXPNO : 9	COMMUNICATE BETWEEN ARDUINO AND RASPBERRY PI USING ANY WIRELESS MEDIUM

AIM:

To Interface and Communication between Raspberry pi and Arduino Nano Using I2C Protocol

REQUIRED ITEMS

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	CS3691 IOT TRAINER KIT • Raspberry Pi Unit • Arduino Nano Unit • LED Array Unit	1
3	Jumper Wires	As Required
4	USB Cable	1

RASPBERRY PI ALGORITHM:

1. Start the program
2. Define I2C Bus and Address
3. if Infinite loop True:
 - Read Data from Interpreter Pane
 - if Data ==1
 - Send 0x1 to Arduino
 - else if Data==0
 - Send 0x0 to Arduino
4. else
 - Stop Program

ARDUINO ALGORITHM:

1. Start the program
2. Define I2C Bus and Address

3. if Infinite loop True:

 Read Data from I2C port

 if Data == 1

 LED ON

 if Data==0

 LED OFF

6. else

 Stop Program

ARDUINO PROGRAM

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27,16,2);
String read_data;
void setup()
{
  Serial.begin(9600);
  lcd.begin();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("WAITING FOR DATA...");
}
void loop()
{
  while (Serial.available())
  {
    delay(10);
    char c = Serial.read();
    read_data+= c;

  }
  if (read_data.length() > 0)
  {
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print(read_data);
    lcd.setCursor(0,1);

    delay(1000);
  }
  read_data="";
  delay(10);
}
RASPBERRY PI PROGRAM
from time import sleep
from i2c_lcd import I2cLcd
from lcd_api import LcdApi
from machine import Pin,I2C
```



```
from machine import Pin, ADC
import time
import machine
import urequests

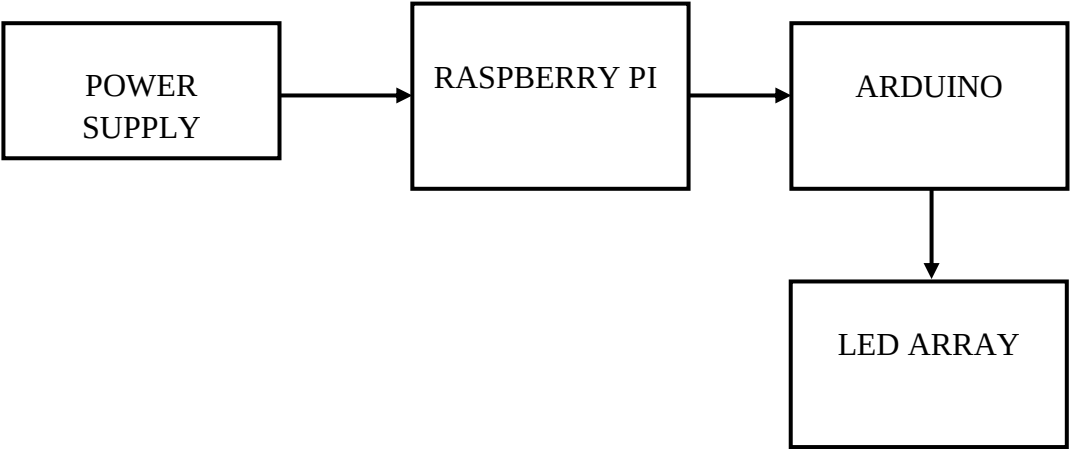
from machine import Pin,UART
uart = UART(0, baudrate=9600, tx=Pin(0), rx=Pin(1))

i2c = I2C(0,scl=Pin(5), sda=Pin(4), freq=100000)#Include the I2C pins
I2C_ADDR = i2c.scan()[0]#Scan I2C addres
lcd = I2cLcd(i2c, I2C_ADDR, 2, 16)#Initialize the I2C module
pot = ADC(Pin(26))
while True:
    lcd.clear()#Clear the display
    pot_value = pot.read_u16()
    print(pot_value)
    lcd.move_to(0,0)#Set the cursor
    lcd.putstr("potvalue - ")
    lcd.putstr(str(pot_value))#Print the distance
    sleep(1)
    uart.write('POT_VALUE:')
    uart.write(str(pot_value))
#    uart.write('\n\r')
    sleep(2)
```

RASPBERRY PI CONNECTION TABLE

S.NO	CONNECTION		PIN MODE
	COMPONENT	RASPBERRY PI	
1	VCC	5V	POWER +
2	GND	GND	POWER -
3	ARDUINO SDA (A4)	SDA (GPIO 2)	I2C DATA
4	ARDUINO SCL (A5)	SCL (GPIO 3)	I2C CLOCK

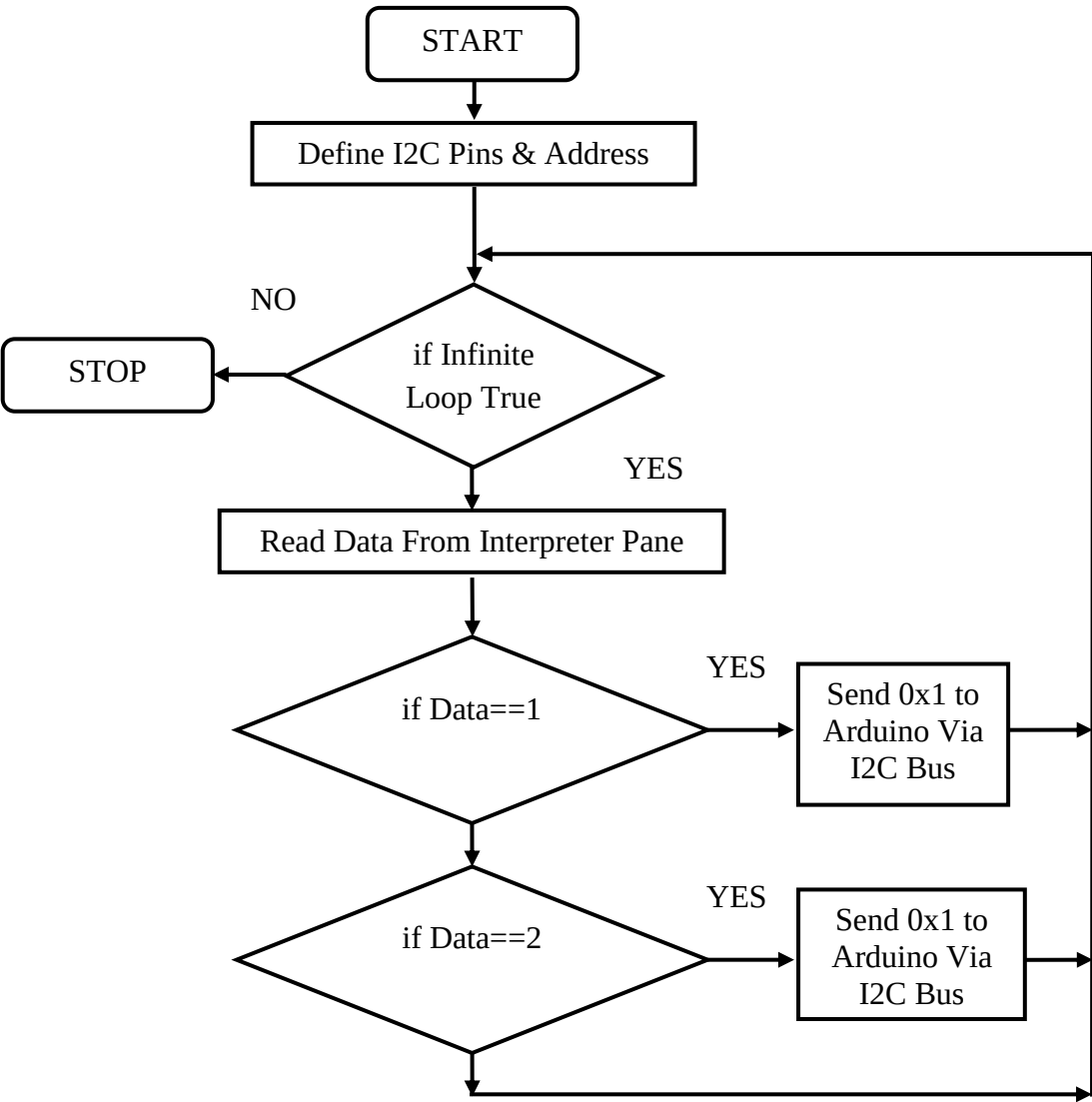
BLOCK DIAGRAM



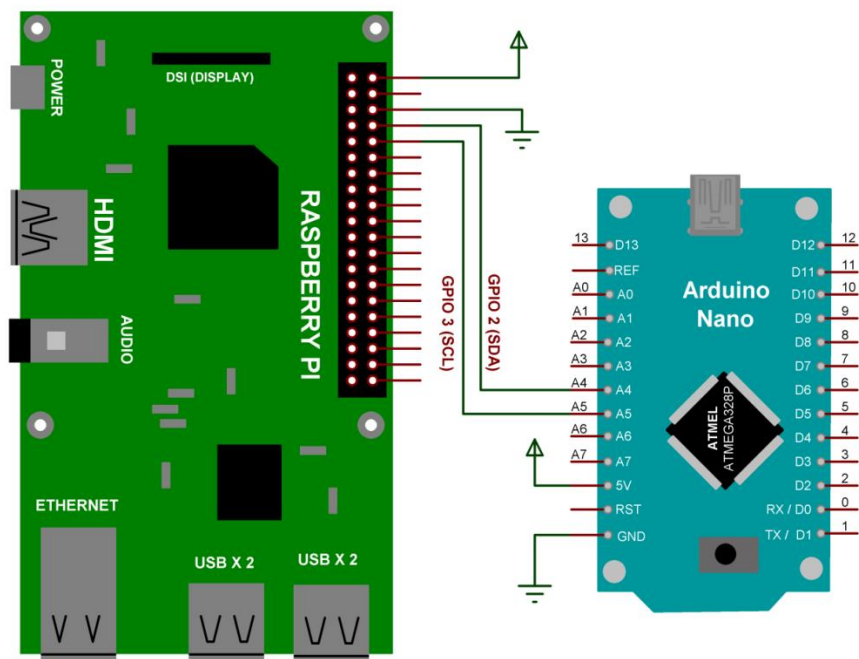
RASPBERRY PI CONNECTION TABLE:

S.NO	CONNECTION		PIN MODE
	COMPONENT	ARDUINO	
1	VCC	5V	POWER +
2	GND	GND	POWER -
3	LED1	D2	OUTPUT

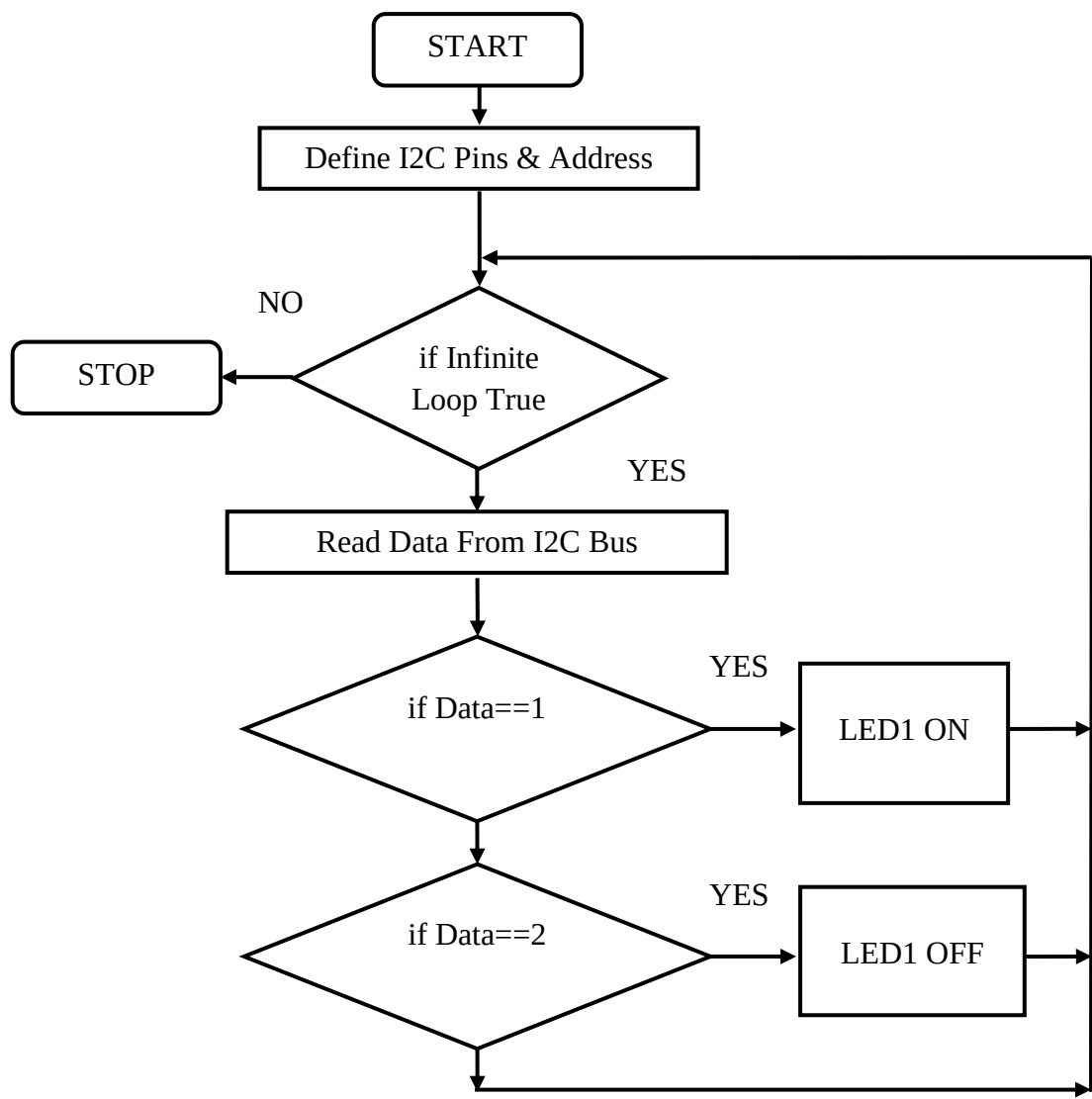
RASPBERRY PI FLOW CHART



CIRCUIT DIAGRAM



ARDUINO FLOW CHART



TERMINAL COMMAND LINE

1. sudo apt-get update
2. sudo apt-get upgrade
3. sudo raspi-config
4. Select Interfacing Options > I2C.
5. Select Yes when prompted to enable the I2C interface.
6. Select Yes when prompted to automatically load the I2C kernel module.
7. Select Finish.
8. i2cdetect -y 1
9. sudo raspi-config
10. Select Interfacing Options > 1 WIRE.
11. Select Yes when prompted to enable the 1 WIRE interface.
12. Select Yes when prompted to automatically load the 1 WIRE kernel module.
13. Select Finish.

PROCEDURE:

1. Start **Thonny** by clicking on the **Raspberry Pi icon** followed by **Programming > Thonny Python IDE**.
2. Write your program in the top pane, click **File > Save** as... to save it,
3. Make hardware connections
4. Click **Run>** Run current script to execute the program. Output will appear in the bottom interpreter pane.

PROCEDURE:

1. Open **ARDUINO IDE** Software
2. Click **File >> New** or **C**
TRL+N to create New Project.
3. To Select Board Click **Tools >> Board > Arduino AVR Boards >> Arduino Nano**
4. To select Processor Click **Tools >> Processor >> ATmega328P (Old Bootloader)**
5. To select Programmer Click **Tools >> Programmer >> AVRISP mkII**
6. Click **File >> Save** or **CTRL+S** to Save Project where we Desire.
7. Type the Program.

8. Click **Sketch >> Verify / Compile** or **CTRL+R** to compile the program (If any error present debug the program)

9. Connect the Arduino Nano with PC / Laptop by USB Cable

10. Select the COM PORT by Click **Tools >> Port** and Select Arduino's COM PORT Number.

Note : If COM PORT Number is Don't Know:

Right Click My Computer Icon and Select Properties

Click Device Manager

Expand Ports (COM & LPT). COM PORT Number will be shown

11. Click **Sketch >> Upload** or **CTRL+U** to upload the program to Arduino Nano.

12. Switch On the corresponding Units we need and Verify the Program.

RESULT:

Thus the Communication between Raspberry pi and Arduino Nano Using I2C Protocol was interfaced successfully

EXPNO: 10	Setup a cloud platform to log the data

AIM:

To Create New Project and Setup Firebase Real-time Database.

REQUIRED ITEMS

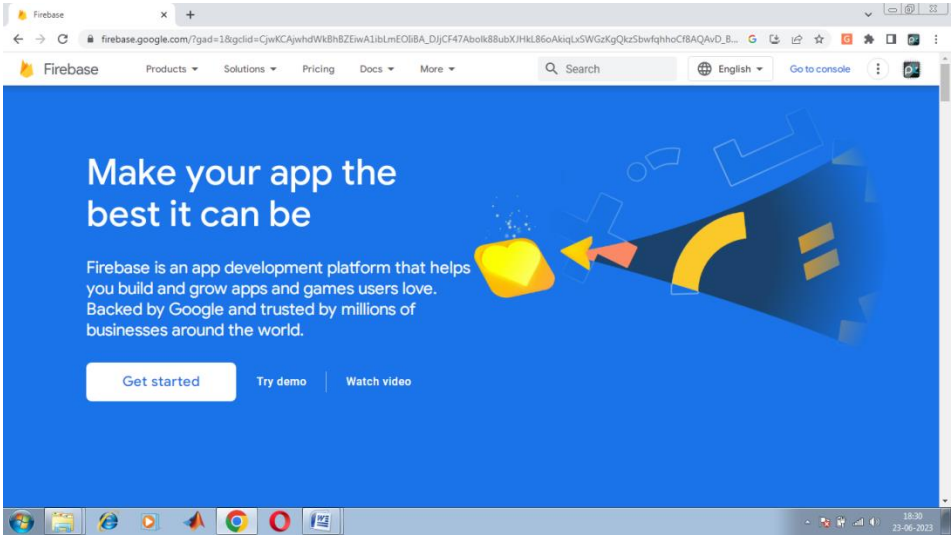
S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: - OS : windows 7 or above - Hard disk : 256 GB or above - RAM : 2 GB or above - Keyboard and Mouse With High Speed Internet Facility	1

PROCEDURE:

STEP_1: First, Visit Firebase Console Using This Visiting The Following URL - <https://console.firebase.google.com>.

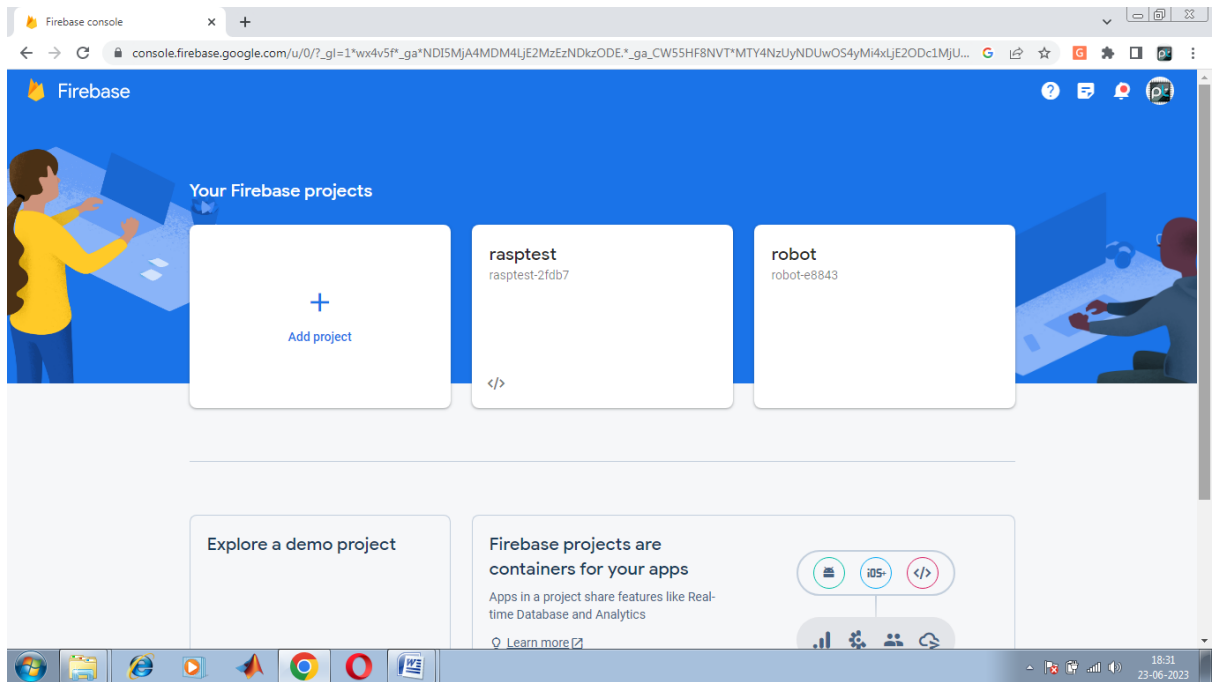
SPEP_2: Login Using Your Google Account - If You Are Not Already Logged In.

STEP_3:



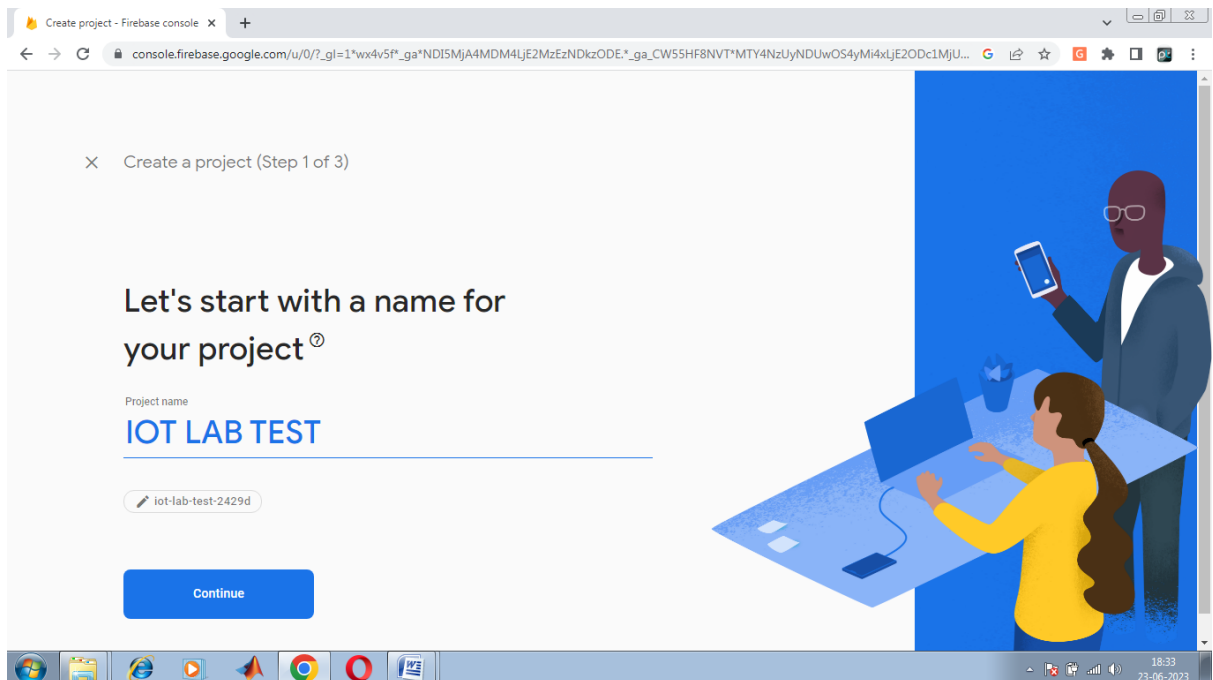
- Click **Get Started**

STEP_4:



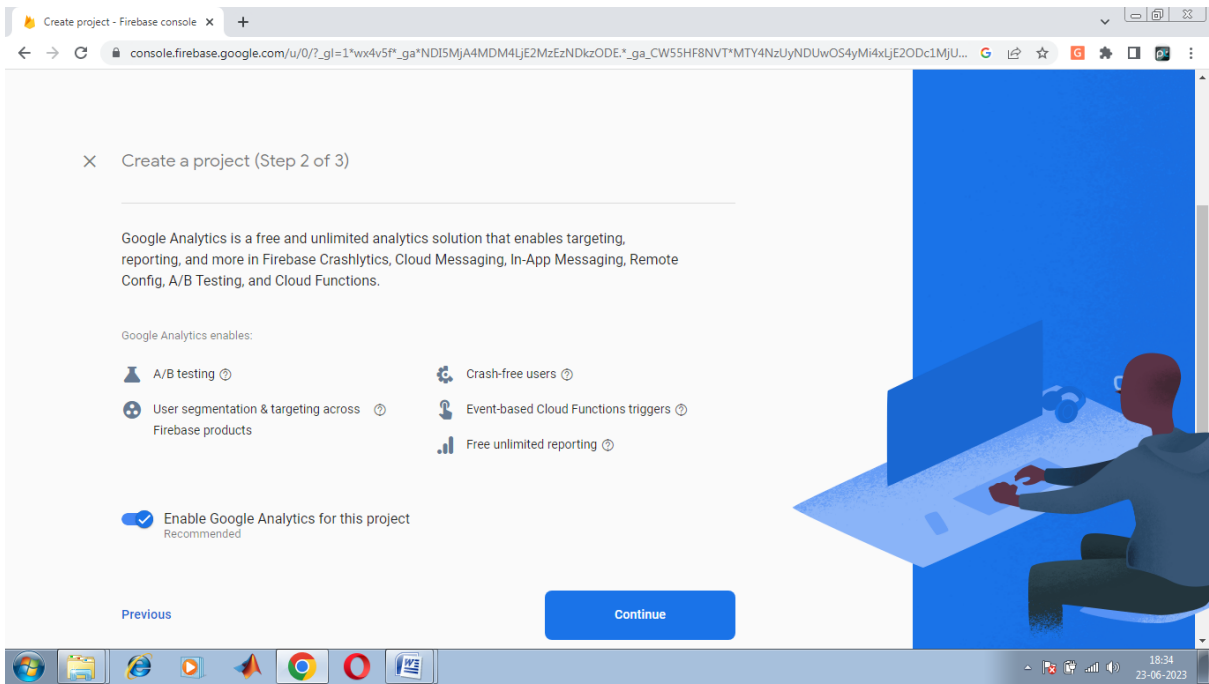
- Click **Add Prjject**

STEP_5:



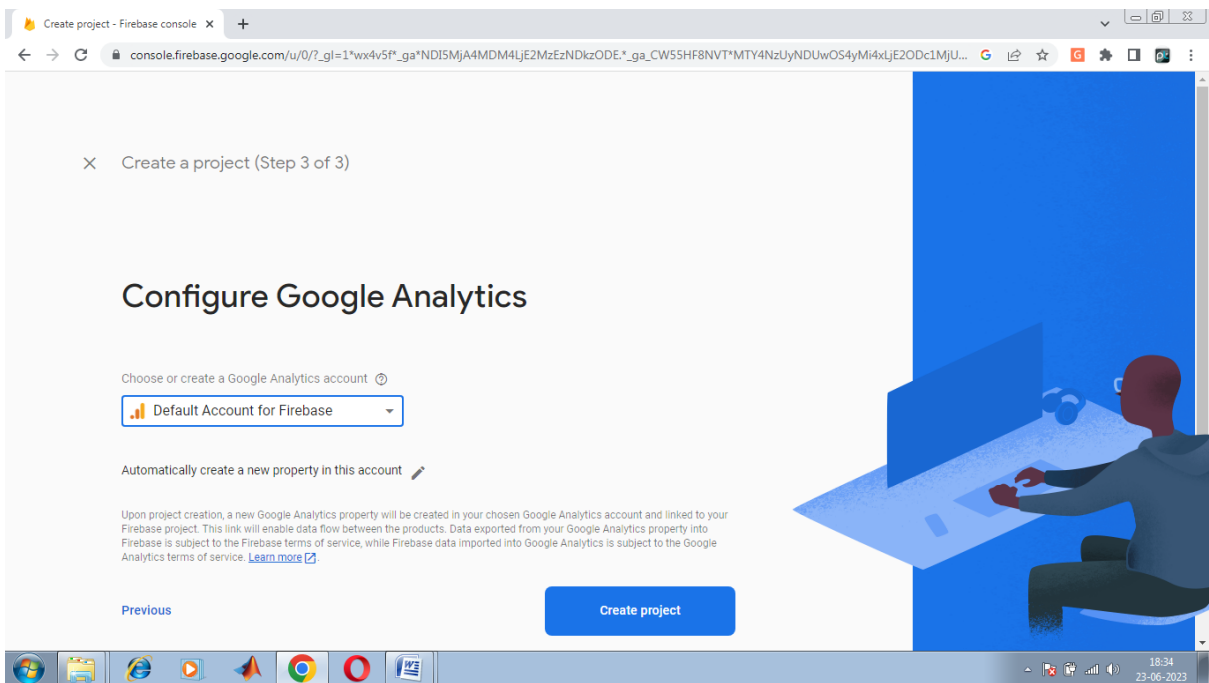
- Enter Your **Project Name** and Click **Continue**

STEP_6:



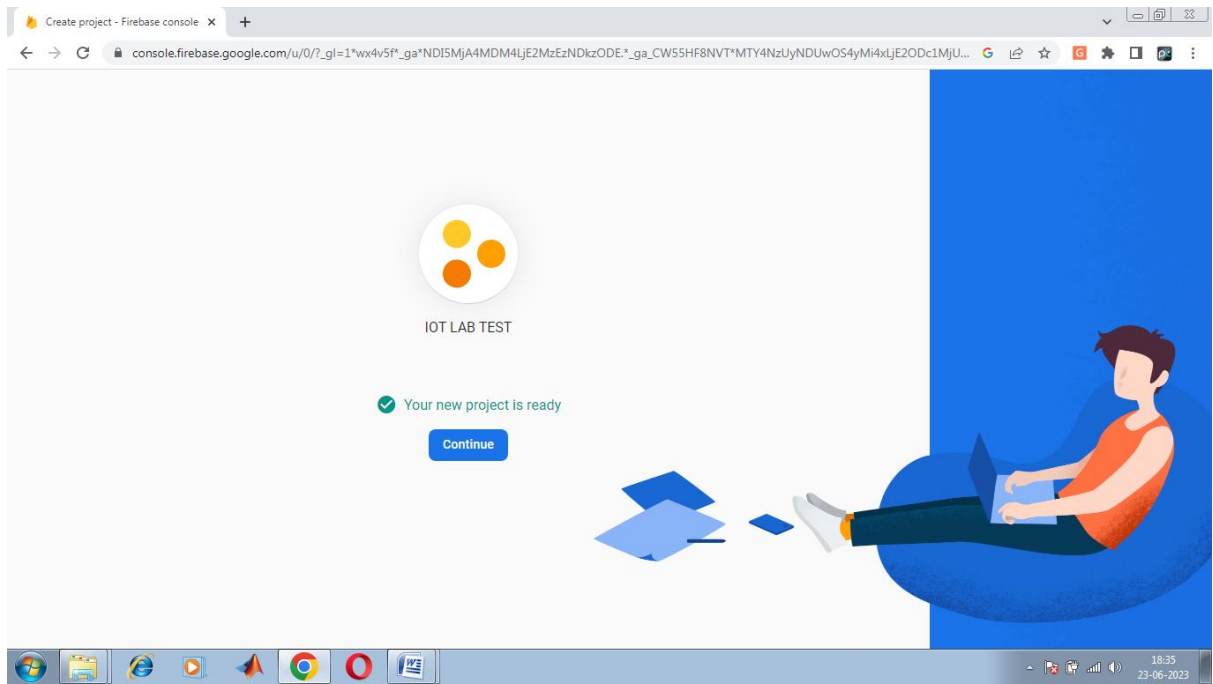
- Click **Continue**

STEP_7:



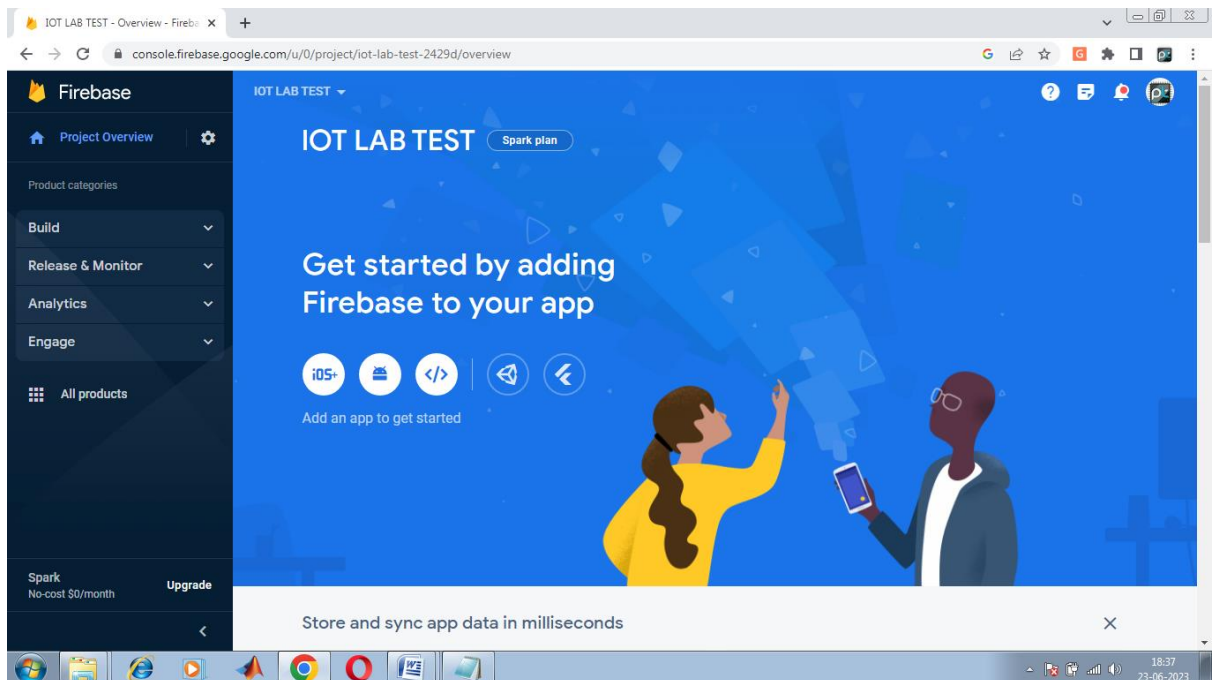
- Select **Default Account For Firebase** And Click **Create Project**

STEP_8:



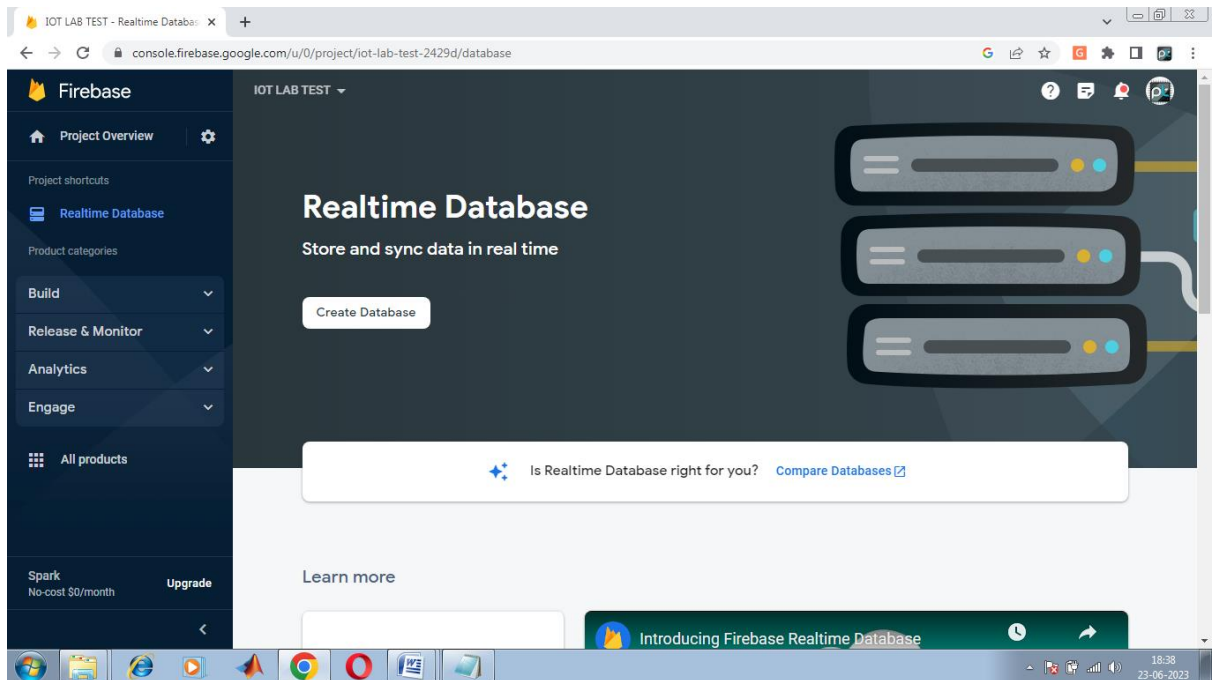
- Project Is Created And Click **Continue**

STEP_9:



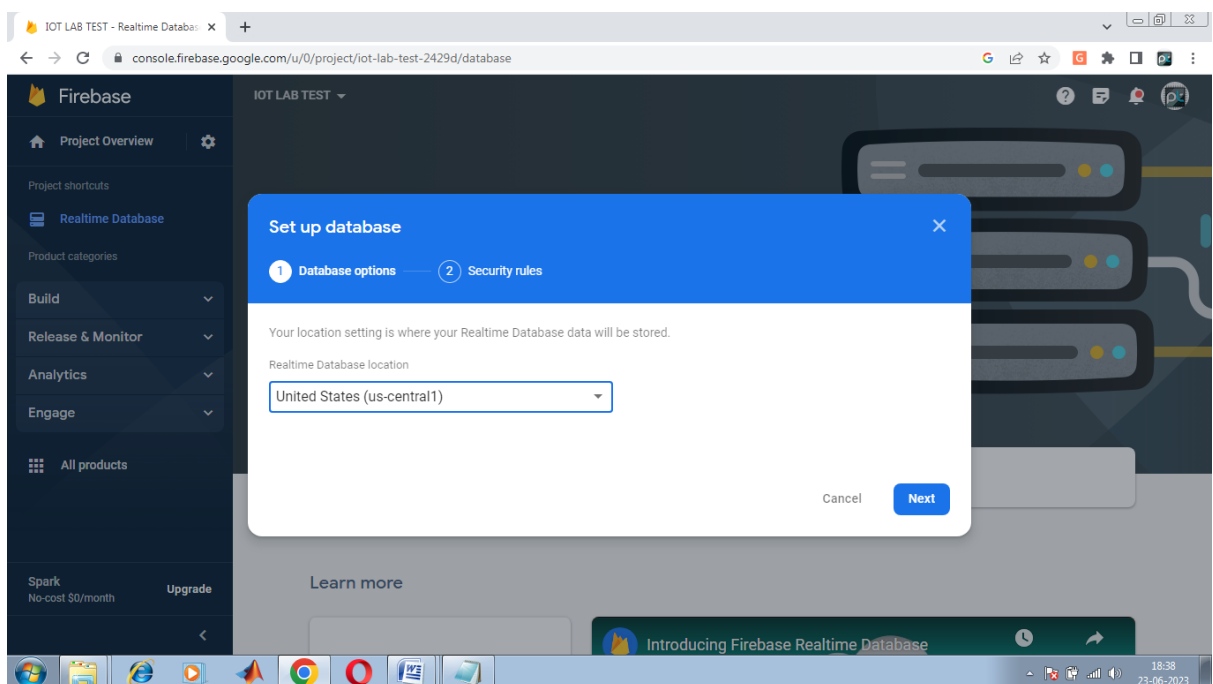
- Firebase Console Home Page Opened and Click **Build And Select Real Time Database**

STEP_10:



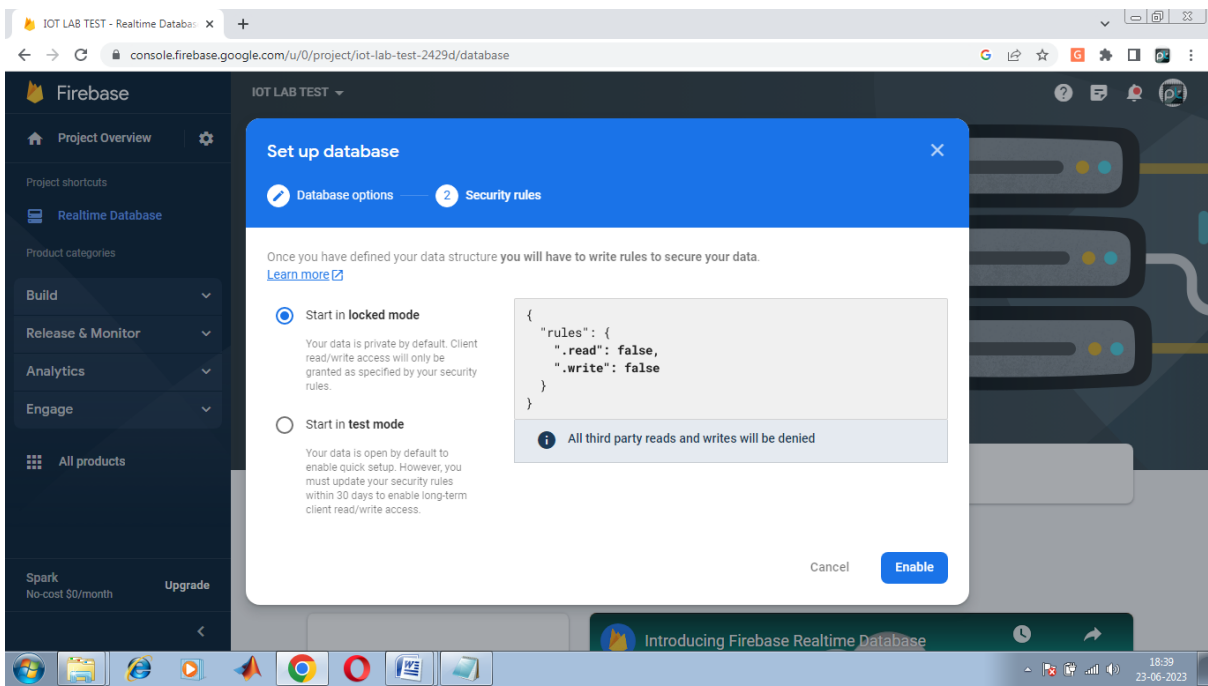
- Click **Create Database**

STEP_11:



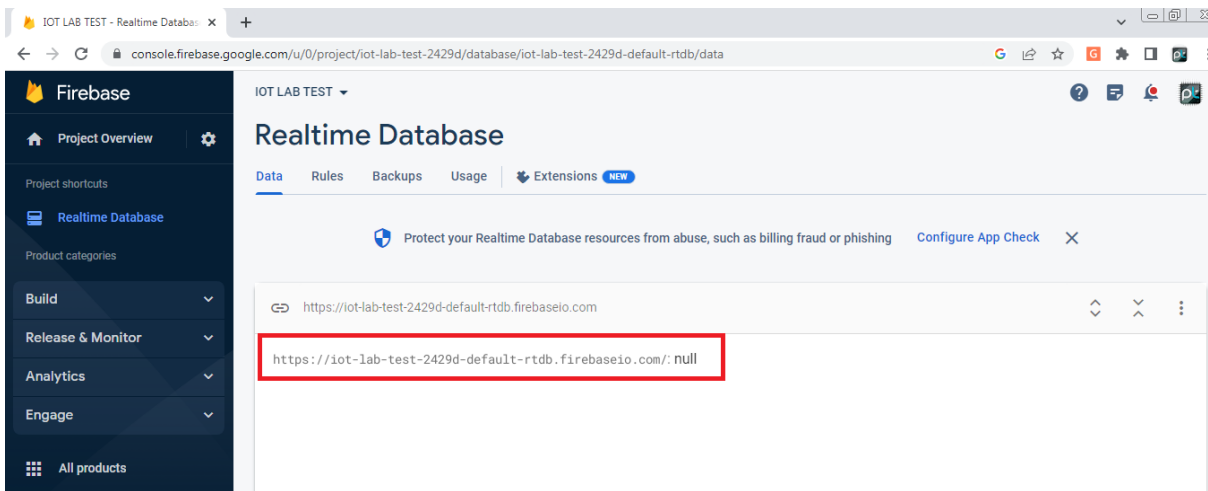
- Select **Realtime Database Location** As **United States (Us-Central1)** And Click **Next**

STEP_12:



- Select **Start In Locked Mode** And Click **Enable**

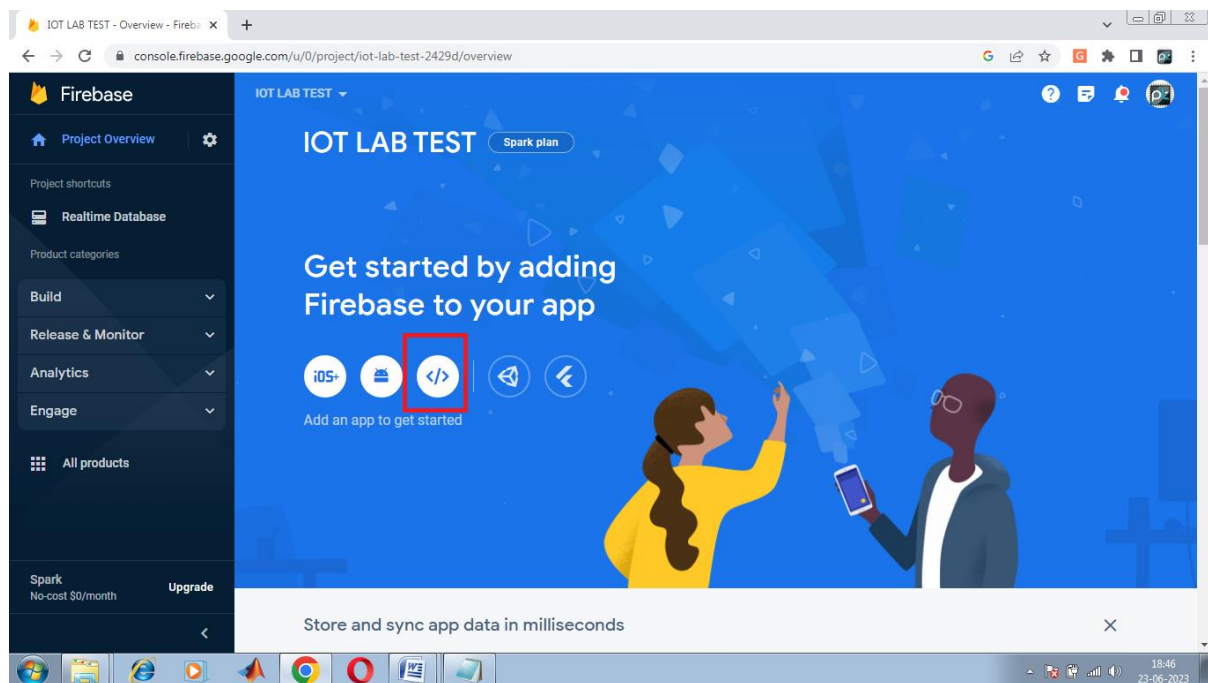
STEP_13:





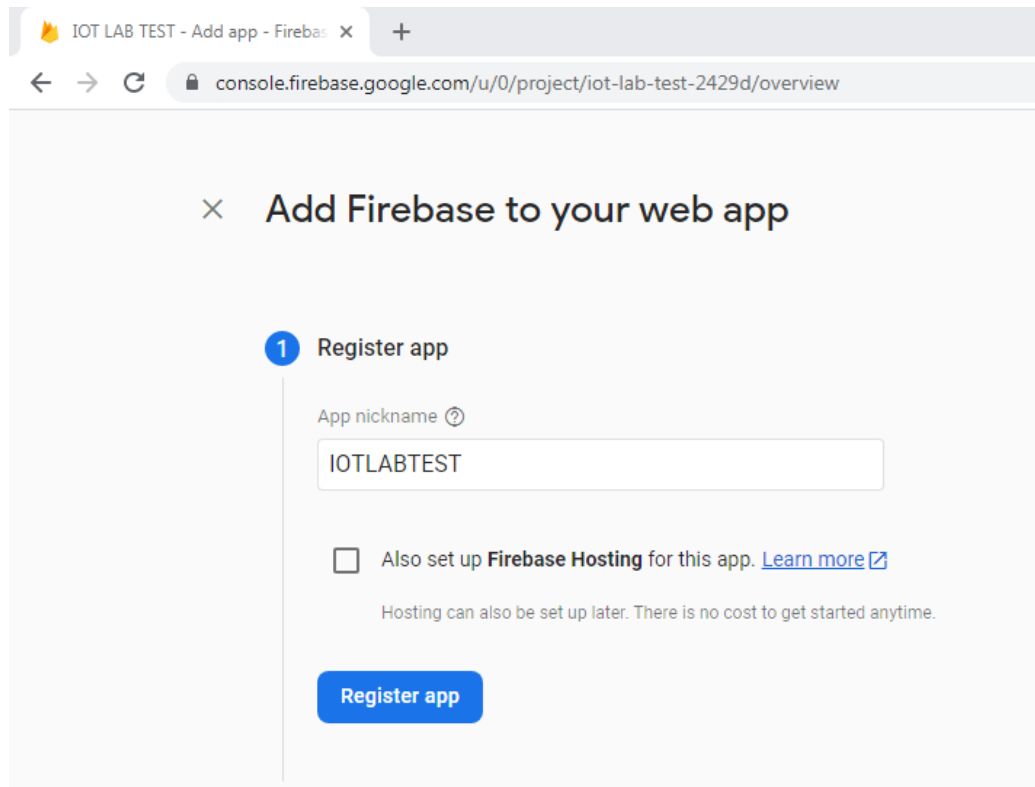
- Now **Real Time Database Is Created** With Host URL. Click Rules And Change **Write And Read Rules As true** And Click **Publish**

STEP_14:



- Click **Project Overview** . Click **Web Link** Mentioned In Red Box

STEP_15:



IOT LAB TEST - Add app - Firebase

console.firebase.google.com/u/0/project/iot-lab-test-2429d/overview

× Add Firebase to your web app

1 Register app

App nickname ⓘ

IOTLABTEST

☐ Also set up **Firebase Hosting** for this app. [Learn more](#)

Hosting can also be set up later. There is no cost to get started anytime.

Register app

- Enter **App Name** And Click **Register App**

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "AIzaSyBU-jPZB6aLXoAI--vQPI8Fr1DV6PR0Q",
  authDomain: "iot-lab-test-2429d.firebaseio.com",
  databaseURL: "https://iot-lab-test-2429d.firebaseio.com",
  projectId: "iot-lab-test-2429d",
  storageBucket: "iot-lab-test-2429d.firebaseio.com",
  messagingSenderId: "1095415346205",
  appId: "1:1095415346205:web:1e861499d1ee8a56ce42f4",
  measurementId: "G-KLJ4EZHJQ9"
};
```

- Copy These Line Saved In Notepad File For Future IOT Project Deployment
- Firebase Configuration Data's Are Created. Click **Continue to Console**

RESULT:

Thus the Cloud log on Data was created successfully

EXPNO: 11	LOG DATA USING RASPBERRY PI AND UPLOAD TO THE CLOUD PLATFORM

AIM:

To Design and Implement IOT Based Weather Station using Raspberry Pi.

REQUIRED ITEMS

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: • OS : windows 7 or above • Hard disk : 256 GB or above • RAM : 2 GB or above • Keyboard and Mouse	1
2	CS3691 IOT TRAINER KIT • Raspberry Pi Unit • I2C LCD Unit • DHT11Sensor Unit	1
3	Jumper Wires	AS Required
4	USB Cable	1

ALGORITHM:

1. Start the program
2. Call GPIO, DHT11, FIREBASE And I2C LCD Library
3. Initialize LCD, DHT11 And FIREBASE
4. Variable Declaration
5. Define GPIO Pins And Its Mode
6. if Infinite loop True:
 - Read Temperature And Humidity Value
 - Clear LCD
 - Print **TEMP: VALUE**
 - Print **HUMI: VALUE**
 - Update Temperature And Humidity Value on IoT Firebase Database
 - if Temperature >=40
 - Print **CLIMATE**
 - Print **SUNNY**
 - Update Climate Status on IoT Firebase Database

```

else if humidity >=80
Print CLIMATE
Print RAINY
Update Climate Status on IoT Firebase Database
else
Print CLIMATE
Print NORMAL
Update Climate Status on IoT Firebase Database

```

7. else

Stop Program

PROGRAM :

```

import lcddriver
from time import *
lcd = lcddriver.lcd()
import RPi.GPIO as GPIO
import pyrebase
from time import sleep
import time
import Adafruit_DHT
import time
DHT_SENSOR = Adafruit_DHT.DHT11
DHT_PIN = 4
prevtemp=30.0
prevhumi=50.0
config = {
    "apiKey": "wgsNjBl7qvzVY0KiezCBplSlRxLI9OiPwEschMfh",
    "authDomain": "rasptest-2fdb7.firebaseio.com",
    "databaseURL": "https://rasptest-2fdb7-default-rtdb.firebaseio.com/",
    "storageBucket": "rasptest123"
}
firebase = pyrebase.initialize_app(config)
GPIO.setmode(GPIO.BCM) # Terminal by number.
GPIO.setwarnings(False)
lcd.lcd_clear()
lcd.lcd_display_string(" IoT Based ", 1)
lcd.lcd_display_string("Weather Station" , 2)
time.sleep(2)
lcd.lcd_clear()
lcd.lcd_display_string(" System", 1)
lcd.lcd_display_string(" *****" , 2)
time.sleep(2)
try:
    while True: # Bucle principal
        database = firebase.database()
        ProjectBucket = database.child("IOTLAB")
        LEDDATA = ProjectBucket.child("LED").get().val()
        humidity, temperature = Adafruit_DHT.read(DHT_SENSOR, DHT_PIN)
        if humidity is not None and temperature is not None:

```

```

        print("Temp={0:0.1f}C Humidity={1:0.1f}%".format(temperature, humidity))
ProjectBucket.child("IOTLAB").child("Humidity").set(humidity)
ProjectBucket.child("IOTLAB").child("Temperature").set(temperature)
lcd lcd_clear()
lcd lcd_display_string("Temp: %d%s C" % (temperature, chr(223)), 1)
    lcd lcd_display_string("Humidity: %d %% " % humidity, 2)
    prevtemp=temperature
    prevhumi=humidity
    time.sleep(2)
else:
    print("DATA READING....");
    lcd lcd_clear()
    lcd lcd_display_string("DATA READING...", 1)
    lcd lcd_display_string("    *****" , 2)
    time.sleep(2)

if prevtemp >=40:
    print("SUNNY")      ProjectBucket.child("IOTLAB").child("Climate").set("Sunny")
    lcd lcd_clear()
    lcd lcd_display_string("    CLIMATE", 1)
    lcd lcd_display_string("    SUNNY" , 2)

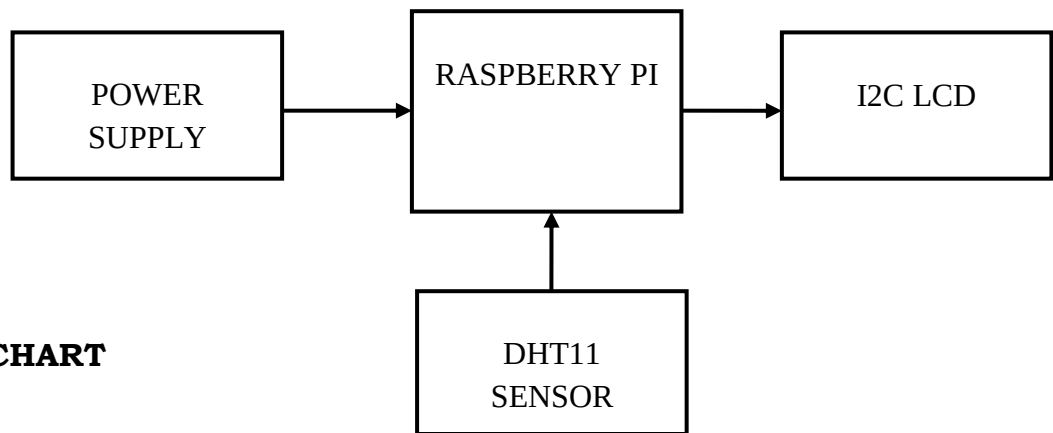
elif prevhumi>=80:
    print("RAINY")
ProjectBucket.child("IOTLAB").child("Climate").set("Rainy")
    lcd lcd_clear()
    lcd lcd_display_string("    CLIMATE", 1)
    lcd lcd_display_string("    RAINY" , 2)
else:
    print("NORMAL")
ProjectBucket.child("IOTLAB").child("Climate").set("Normal")
    lcd lcd_clear()
    lcd lcd_display_string("    CLIMATE", 1)
    lcd lcd_display_string("    NORMAL" , 2)
    time.sleep(2)
except KeyboardInterrupt:
    GPIO.cleanup()

```

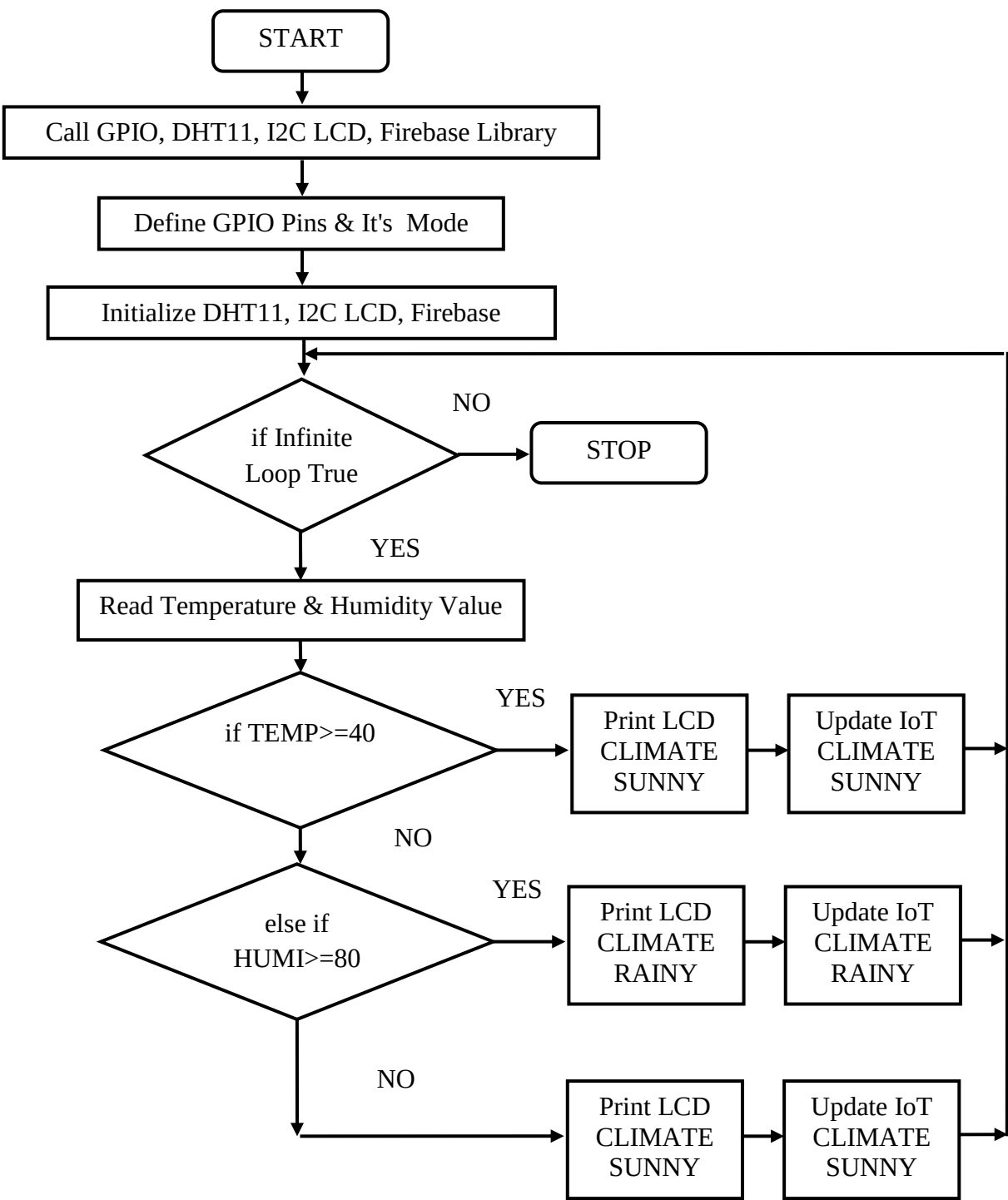
CONNECTION TABLE:

S.NO	CONNECTION		PIN MODE
	COMPONENT	RASPBERRY PI	
1	VCC	5V	POWER +
2	GND	GND	POWER -
3	LCD (SDA)	GPIO 2	Serial Data
4	LCD (SCL)	GPIO 3	Serial Clock
5	DHT11	GPIO 25	Digital Input

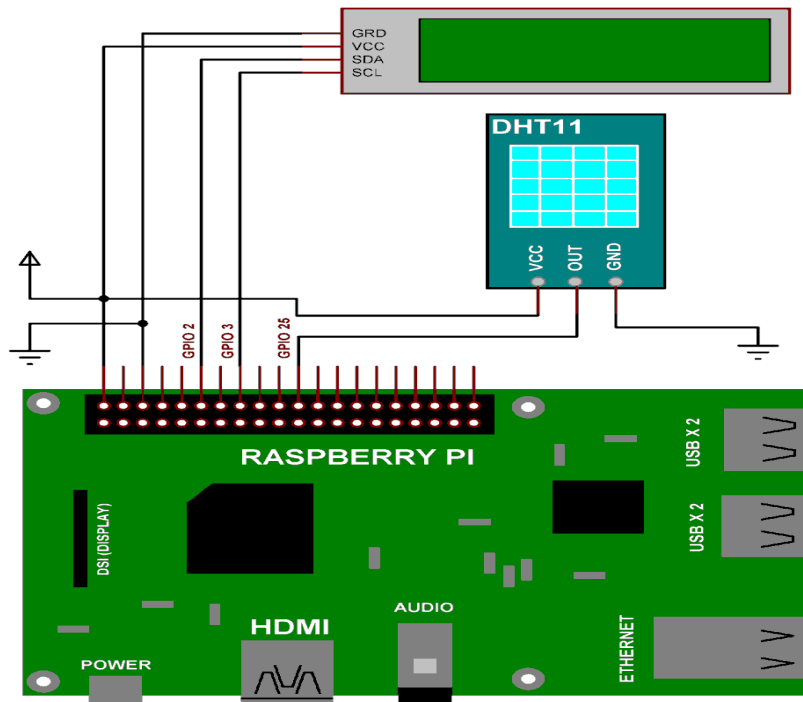
BLOCK DIAGRAM



FLOW CHART



CIRCUIT DIAGRAM



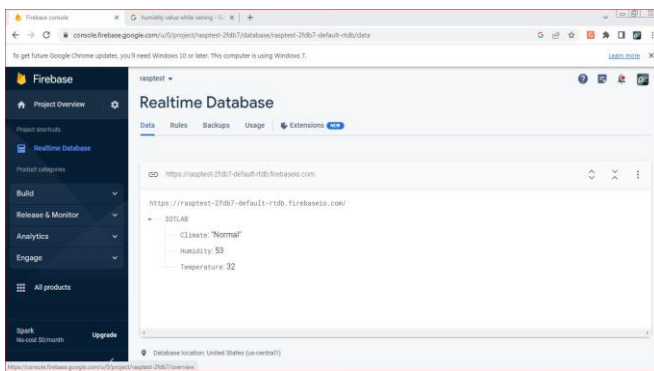
TERMINAL COMMAND LINE

1. `sudo apt-get update`
2. `sudo apt-get upgrade`
3. `sudo raspi-config`
4. Select Interfacing Options > I2C.
5. Select Yes when prompted to enable the I2C interface.
6. Select Yes when prompted to automatically load the I2C kernel module.
7. Select Finish.
8. `i2cdetect -y 1`
9. `sudo raspi-config`
10. Select Interfacing Options > 1 WIRE.
11. Select Yes when prompted to enable the 1 WIRE interface.
12. Select Yes when prompted to automatically load the 1 WIRE kernel module.
13. Select Finish.
14. `sudo apt-get update.`
15. `sudo apt-get install python-dev.`
16. `sudo python get-pip OR sudo apt-get install python-pip (new Raspbian versions)`
17. `sudo pip install pyrebase.`

PROCEDURE:

1. Start **Thonny** by clicking on the **Raspberry Pi icon** followed by **Programming > Thonny Python IDE**.
2. Write your program in the top pane, click **File > Save** as... to save it,
3. Make hardware connections
4. Click **Run>** Run current script to execute the program. Output will appear in the bottom interpreter pane.

RESULT SCREEN SHOT



RESULT:

Thus the data was uploaded successfully in the Cloud by using Raspberry Pi

EXPNO: 12	DESIGN AN IOT BASED SYSTEM

Design And Implementation Of IoT Based Home Automation Using Raspberry Pi

AIM:

To Design and Implement IOT Based Home Automation using Raspberry Pi.

REQUIRED ITEMS

S.NO	ITEM	QTY
1	PC / LAPTOP SYSTEM SPECIFICATION: - OS : windows 7 or above - Hard disk : 256 GB or above - RAM : 2 GB or above - Keyboard and Mouse	1
2	CS3691 IOT TRAINER KIT - Raspberry Pi Unit - Relay Unit	1
3	Jumper Wires	AS Required
4	USB Cable	1

ALGORITHM:

1. Start the program
2. Call GPIO, FIREBASE And I2C LCD Library
3. Initialize LCD, And FIREBASE
4. Variable Declaration
5. Define GPIO Pins And Its Mode
6. if Infinite loop True:
 - Read Firebase Data
 - if LED1Data =1
 - Relay_1 ON
 - else
 - Relay_1 OFF
 - if LED2Data =1
 - Relay_2 ON
 - else
 - Relay_2 OFF
 - Clear LCD
 - Print **Relay_1: Status**
 - Print **Relay_2: Status**
 - Update Relay_1 & Relay_2 Status on IoT Firebase Database
7. else
 - Stop Program

PROGRAM

```
import lcddriver
from time import *
lcd = lcddriver.lcd()
import busio
import RPi.GPIO as GPIO
import pyrebase
from time import sleep
import time

config = {
    "apiKey": "wgsNjBl7qvzVY0KiezCBp1SlRxLI9OiPwEschMfh",
    "authDomain": "rasptest-2fdb7.firebaseio.com",
    "databaseURL": "https://rasptest-2fdb7-default-rtdb.firebaseio.com/",
    "storageBucket": "rasptest123"
}

firebase = pyrebase.initialize_app(config)

LED_1 = 18      # Terminal by number, el 22 es el GPIO25.
LED_2 = 23

GPIO.setmode(GPIO.BCM) # Terminal by number.
GPIO.setwarnings(False)
GPIO.setup(LED_1,GPIO.OUT)
GPIO.setup(LED_2,GPIO.OUT)

lcd.lcd_clear()
lcd.lcd_display_string(" IoT Based ", 1)
lcd.lcd_display_string("Home Automation" , 2)
time.sleep(2)
lcd.lcd_clear()
lcd.lcd_display_string(" System", 1)
lcd.lcd_display_string(" *****" , 2)
time.sleep(2)
try:
    while True:      # Bucle principal

        database = firebase.database()
        ProjectBucket = database.child("IOTLAB")
        LEDDATA1 = ProjectBucket.child("LED1").get().val()
        ProjectBucket = database.child("IOTLAB")
        LEDDATA2 = ProjectBucket.child("LED2").get().val()
        print(LEDDATA2)
        time.sleep(0.5)
        lcd.lcd_clear()

        if str(LEDDATA1) == "1":
            print("LED_1 now is ON.")
            GPIO.output(LED_1, GPIO.HIGH)
            ProjectBucket.child("IOTLAB").child("L1").set("ON")
            lcd.lcd_display_string("LED_1 : ON", 1)
        else:
            print("LED_1 now is OFF.")
            GPIO.output(LED_1, GPIO.LOW)
            ProjectBucket.child("IOTLAB").child("L1").set("OFF")
```

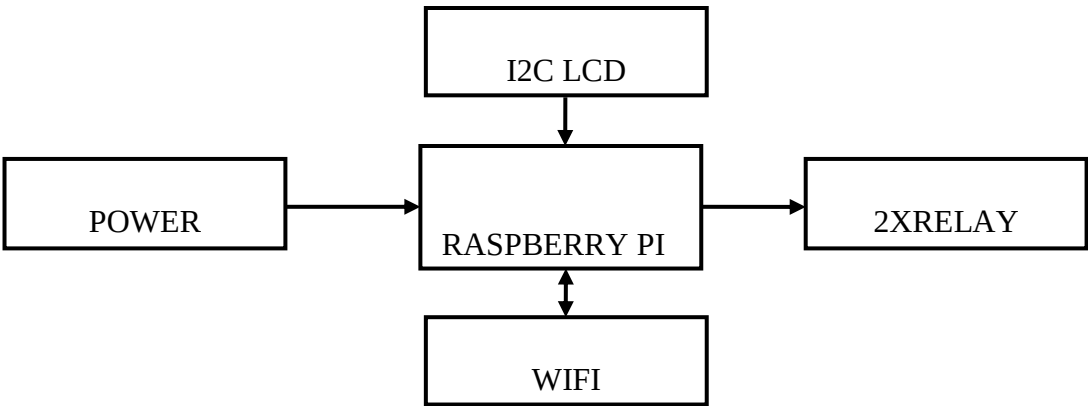
```
lcd lcd_display_string("LED_1 : OFF", 1)

if str(LEDDATA2) == "1":
    print("LED_2 now is ON.")
    GPIO.output(LED_2, GPIO.HIGH)
    ProjectBucket.child("IOTLAB").child("L2").set("ON")
    lcd lcd_display_string("LED_2 : ON", 2)
else:
    print("LED_2 now is OFF.")
    GPIO.output(LED_2, GPIO.LOW)
    ProjectBucket.child("IOTLAB").child("L2").set("OFF")
    lcd lcd_display_string("LED_2 : OFF", 2)
time.sleep(2)
except KeyboardInterrupt:
GPIO.cleanup()
```

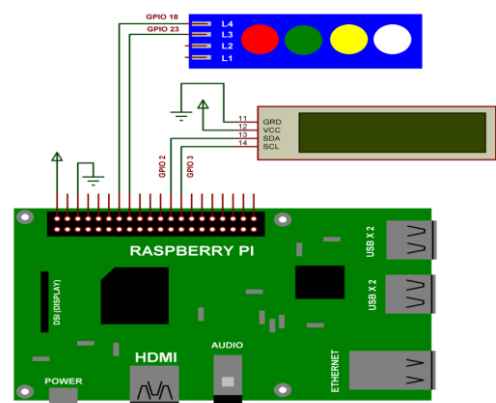
CONNECTION TABLE

S.NO	CONNECTION		PIN MODE
	COMPONENT	RASPBERRY PI	
1	VCC	5V	POWER +
2	GND	GND	POWER -
3	Relay_1 (R1)	GPIO 18	OUTPUT
4	Relay_2 (R2)	GPIO 23	OUTPUT
5	LCD (SDA)	GPIO 2	Serial Data
6	LCD (SCL)	GPIO 3	Serial Clock

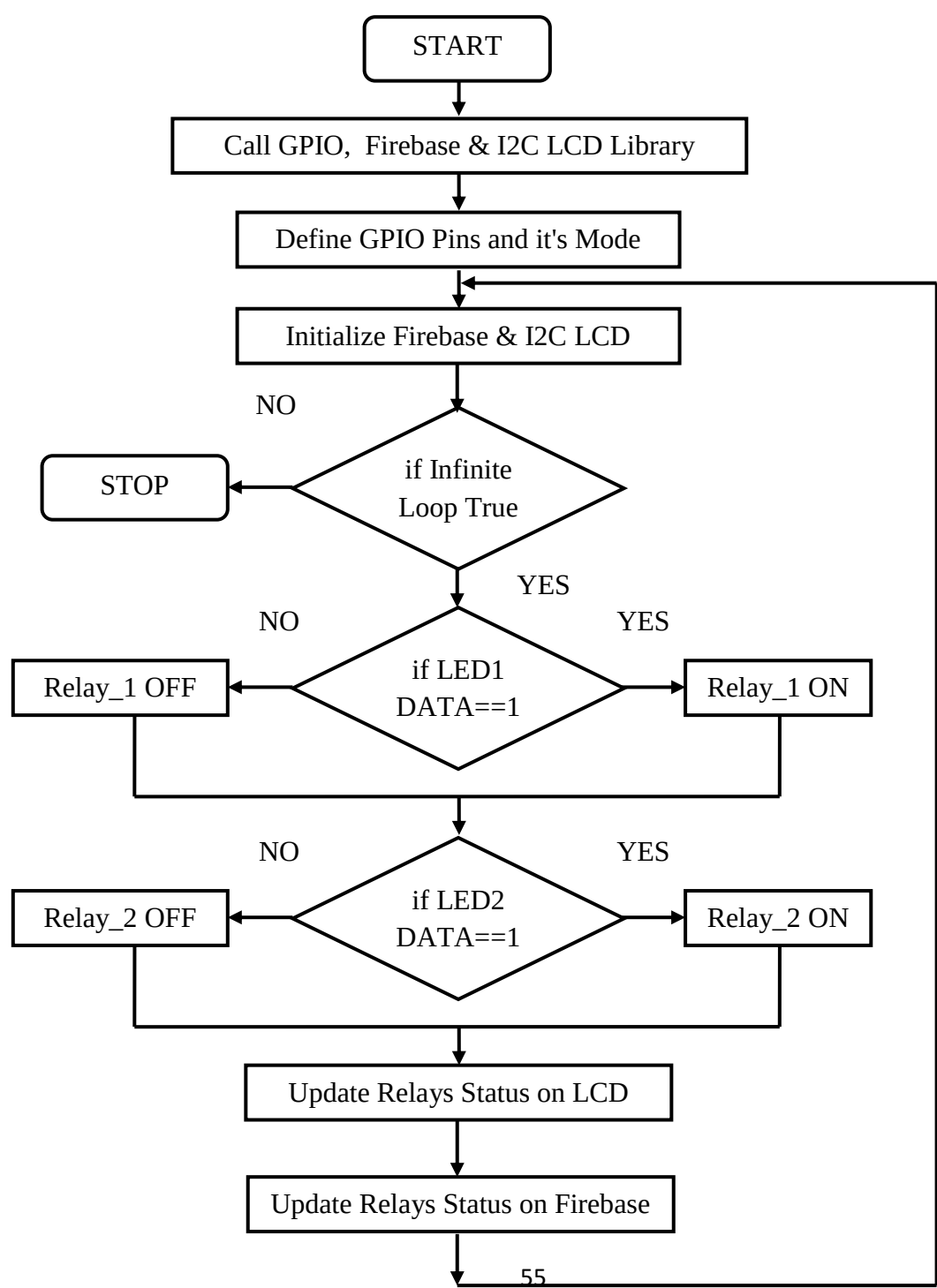
BLOCK DIAGRAM



CIRCUIT DIAGRAM



FLOW CHART



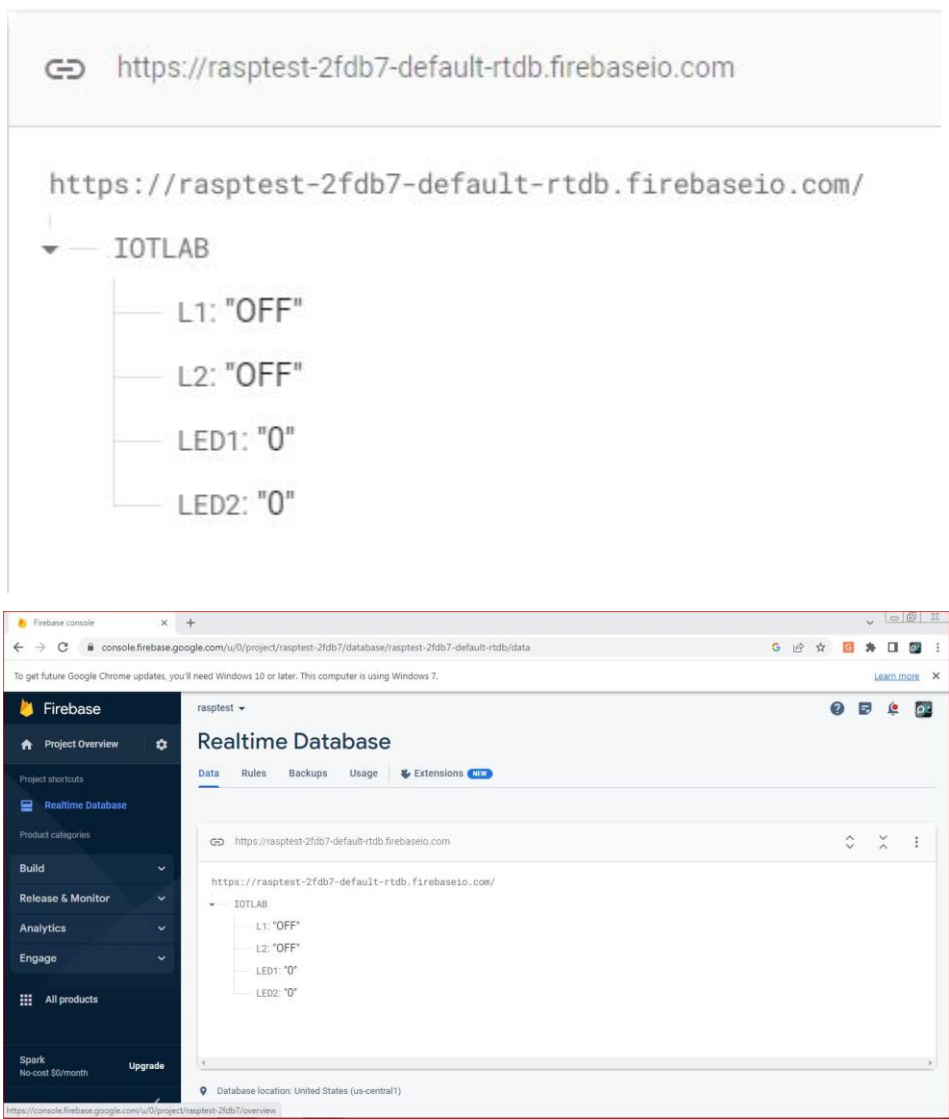
TERMINAL COMMAND LINE

1. `sudo apt-get update`
2. `sudo apt-get upgrade`
3. `sudo raspi-config`
4. Select Interfacing Options > I2C.
5. Select Yes when prompted to enable the I2C interface.
6. Select Yes when prompted to automatically load the I2C kernel module.
7. Select Finish.
8. `i2cdetect -y 1`
9. `sudo raspi-config`
10. Select Interfacing Options > 1 WIRE.
11. Select Yes when prompted to enable the 1 WIRE interface.
12. Select Yes when prompted to automatically load the 1 WIRE kernel module.
13. Select Finish.
14. `sudo apt-get update.`
15. `sudo apt-get install python-dev.`
16. `sudo python get-pip` OR `sudo apt-get install python-pip` (new Raspian versions)
17. `sudo pip install pyrebase.`

PROCEDURE:

1. Start **Thonny** by clicking on the **Raspberry Pi icon** followed by **Programming > Thonny Python IDE.**
2. Write your program in the top pane, click **File > Save** as... to save it,
3. Make hardware connections
4. Click **Run>** Run current script to execute the program. Output will appear in the bottom interpreter pane.

RESULT SCREEN SHOT



RESULT:

Thus the Implementation of IoT Based Home Automation Using Raspberry was Successfully Designed